



Pashov Audit Group

Polygun Security Review

April 16th 2026 - April 22nd 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About Polygon	4
6. Executive Summary	4
7. Findings	5
High findings	8
[H-01] Receipt fetch failure after USDC transfer allows repeated commission payouts	8
[H-02] <code>resetPendingCommission</code> returns zero instead of user pending commission value	11
Medium findings	16
[M-01] Fee transfer dedup relies on migration only unique index not in schema	16
[M-02] Promo can be stuck in <code>processing</code> state due to missing revert logic	16
[M-03] <code>revertPromoToPending</code> allows reversion after successful onchain transfer	18
[M-04] Migrations hold ACCESS EXCLUSIVE on large tables without timeout or concurrency settings	20
[M-05] <code>updateFill</code> lacks idempotency guard leading to inflated <code>filledSize</code> in orders	24
[M-06] Missing unique constraint on <code>fee_transfers</code> leads to ineffective deduplication	26
[M-07] Referral percent override cannot be applied under the current user creation flow	27
Low findings	30
[L-01] Referral chain lookups allow banned users to accrue commissions	30
[L-02] <code>users.telegramId</code> lacks unique index in <code>schema.ts</code> risking data integrity	30
[L-03] Missing state update leaves wallet creation flag incorrect	30
[L-04] <code>updateLastScannedBlock</code> overwrites block number causing concurrency issues	31
[L-05] Migration ledger abandoned with schema of record from <code>drizzle-kit</code> push output	34
[L-06] <code>findByTokenSuffix</code> uses unvalidated input for leading wildcard LIKE on nonindexed column	35
[L-07] ResumeFollow does not clear auto-stop fields after manual resume in Telegram UI	37
[L-08] Max subscriptions per user not enforced due to race condition in follow method	39
[L-09] <code>recordCopyTrade</code> lacks idempotency key causing duplicate event in aggregate	41
[L-10] <code>getFollowById</code> and <code>toggleNotificationsMuted</code> do not filter soft-deleted follows	43
[L-11] Referral_commissions has no UNIQUE on (trade_id beneficiary_id level)	44
[L-12] Self-follow guard only fires when both parties are in-protocol users	46
[L-13] Copy trading follows unique index lack where deleted_at is null	48
[L-14] Record copy trade error handler spreads drizzle internals to stdout	50
[L-15] Asymmetric soft-delete in <code>getReferralChain</code> disrupts F3 commission payouts	52
[L-16] Deposit checker overlapping runs can send duplicate reminders	53
[L-17] ClaimPromo string comparison can fail after transfer leading to replay loss risk	55
[L-18] Fee_transfers.trade_type is unconstrained text type safety is TS only	56
[L-19] <code>isActive</code> not validated in <code>createUserPromo</code> or <code>claimPromo</code> after admin deactivation	58
[L-20] Unbounded <code>markWithdrawn</code> function marks all credited commissions as withdrawn	61
[L-21] Case-sensitive <code>findBySafeAddress</code> fails on checksummed addresses in database	62
[L-22] Relayer execution accepts unverified safe address	63



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Polygon

Polygon is a copy trading platform built on top of Polymarket that enables users to automatically mirror trades from key opinion leaders. It provides wallet management, limit orders, fee processing, referral tracking, and commission distribution for prediction market trading.

6. Executive Summary

A time-boxed security review of the `toantt208/polygon_audit` repository was done by Pashov Audit Group, during which `Oxaudron`, `Rayaa`, `Shurikenzer`, `naman`, `5m477` engaged to review Polygon. A total of 31 issues were uncovered.

Protocol Summary

Project Name	Polygon
Protocol Type	Polymarket copy trading
Timeline	April 16th 2026 - April 22nd 2026

Review commit hash:

- [3cd6ba3f5afa13e77efbcfa9bf2bb74d51ac0297](#)
(toantt208/polygon_audit)

Fixes review commit hash:

- [7fc47592883213d3cdab622f9f8759cbd828059e](#)
(toantt208/polygon_audit)

Scope

`commissions.ts` `copy-trading.ts` `fee-errors.ts` `fee-transfers.ts` `index.ts`
`kol-promo.ts` `limit-orders.ts` `market-cache.ts` `referrals.ts` `scanner-`
`state.ts` `schema.ts` `temporary-wallets.ts` `truncate-decimal.ts` `users.ts`
`wallets.ts` `deploy-safe.ts`



7. Findings

Findings count

Severity	Amount
High	2
Medium	7
Low	22
Total findings	31

Summary of findings

ID	Title	Severity	Status
[H-01]	Receipt fetch failure after USDC transfer allows repeated commission payouts	High	Resolved
[H-02]	<code>resetPendingCommission</code> returns zero instead of user pending commission value	High	Resolved
[M-01]	Fee transfer dedup relies on migration only unique index not in schema	Medium	Resolved
[M-02]	Promo can be stuck in <code>processing</code> state due to missing revert logic	Medium	Resolved
[M-03]	<code>revertPromoToPending</code> allows reversion after successful onchain transfer	Medium	Resolved
[M-04]	Migrations hold ACCESS EXCLUSIVE on large tables without timeout or concurrency settings	Medium	Resolved
[M-05]	<code>updateFill</code> lacks idempotency guard leading to inflated <code>filledSize</code> in orders	Medium	Resolved
[M-06]	Missing unique constraint on <code>fee_transfers</code> leads to ineffective deduplication	Medium	Resolved
[M-07]	Referral percent override cannot be applied under the current user creation flow	Medium	Resolved
[L-01]	Referral chain lookups allow banned users to accrue commissions	Low	Resolved
[L-02]	<code>users.telegramId</code> lacks unique index in <code>schema.ts</code> risking data integrity	Low	Resolved



ID	Title	Severity	Status
[L-03]	Missing state update leaves wallet creation flag incorrect	Low	Resolved
[L-04]	<code>updateLastScannedBlock</code> overwrites block number causing concurrency issues	Low	Resolved
[L-05]	Migration ledger abandoned with schema of record from <code>drizzle-kit</code> push output	Low	Acknowledged
[L-06]	<code>findByTokenSuffix</code> uses unvalidated input for leading wildcard LIKE on nonindexed column	Low	Resolved
[L-07]	ResumeFollow does not clear auto-stop fields after manual resume in Telegram UI	Low	Resolved
[L-08]	Max subscriptions per user not enforced due to race condition in follow method	Low	Resolved
[L-09]	<code>recordCopyTrade</code> lacks idempotency key causing duplicate event in aggregate	Low	Resolved
[L-10]	<code>getFollowById</code> and <code>toggleNotificationsMuted</code> do not filter soft-deleted follows	Low	Resolved
[L-11]	Referral_commissions has no UNIQUE on (trade_id beneficiary_id level)	Low	Resolved
[L-12]	Self-follow guard only fires when both parties are in-protocol users	Low	Resolved
[L-13]	Copy trading follows unique index lack where deleted_at is null	Low	Resolved
[L-14]	Record copy trade error handler spreads drizzle internals to stdout	Low	Resolved
[L-15]	Asymmetric soft-delete in <code>getReferralChain</code> disrupts F3 commission payouts	Low	Resolved
[L-16]	Deposit checker overlapping runs can send duplicate reminders	Low	Resolved
[L-17]	ClaimPromo string comparison can fail after transfer leading to replay loss risk	Low	Resolved
[L-18]	Fee_transfers.trade_type is unconstrained text type safety is TS only	Low	Resolved



ID	Title	Severity	Status
[L-19]	<code>isActive</code> not validated in <code>createUserPromo</code> or <code>claimPromo</code> after admin deactivation	Low	Resolved
[L-20]	Unbounded <code>markWithdrawn</code> function marks all credited commissions as withdrawn	Low	Resolved
[L-21]	Case-sensitive <code>findBySafeAddress</code> fails on checksummed addresses in database	Low	Resolved
[L-22]	Relayer execution accepts unverified safe address	Low	Resolved



High findings

[H-01] Receipt fetch failure after USDC transfer allows repeated commission payouts

Severity

Impact: High

Likelihood: Medium

Description

The referral-withdraw worker sends USDC from the master Safe to the user, then separately fetches the on-chain receipt to verify settlement. The `transferSucceeded` flag is only flipped **after** both the transfer and the receipt check succeed. If the receipt fetch throws or returns null between the two steps, the catch block treats the whole operation as failed and calls `addPendingCommission` to restore the user's balance even though the USDC has already landed on-chain.

```
// packages/queue/src/workers/referral-withdraw.worker.ts:89-171
// Execute transfer from master's Gnosis Safe to user's wallet
const result = await sdk.transferUsdc({
  to: destinationAddress,
  amount: amountInUnits,
});

if (!result.success) {
  throw new Error(`USDC transfer failed: state=${result.state}, txHash=${
result.transactionHash || 'none'}`);
}

const txHash = result.transactionHash || '';
if (!txHash) {
  throw new Error('Transfer returned no transaction hash');
}

// Verify on-chain transaction receipt
const rpcUrl = getConfig().RPC_PROVIDER_URL;
const provider = new JsonRpcProvider(rpcUrl);
const receipt = await provider.getTransactionReceipt(txHash);
if (!receipt || receipt.status !== 1) {
  throw new Error(`Transaction failed on-chain: txHash=${txHash}, status=${receipt?.status}
`);
}

// Mark transfer as succeeded; do NOT restore commission after this point
transferSucceeded = true;
...

} catch (error) {
```




```
...
// Only restore the pending commission if the USDC transfer did NOT succeed
if (!transferSucceeded) {
  try {
    await userService.addPendingCommission(userId, amount);
  }
  ...
}
```

The enabler primitives live in the database layer. `resetPendingCommission` (users.ts:397-419) atomically snapshots and zeros the balance at enqueue time. `addPendingCommission` (users.ts:377-391) restores by addition. `commissions.markWithdrawn` is only reached after `transferSucceeded = true`, so a failure before that assignment leaves the underlying `referral_commissions` rows in `credited` state forever; they never transition to `withdrawn` despite USDC having been paid out.

```
// packages/database/src/services/users.ts:377-391 (addPendingCommission)
async addPendingCommission(userId: string, amount: number): Promise<void> {
  if (amount <= 0) return;
  try {
    await this.db.update(users)
      .set({
        pendingCommission: sql`COALESCE(${users.pendingCommission}, 0) + ${
          amount.toString()
       }`,
        updatedAt: new Date(),
      })
      .where(eq(users.id, userId));
    ...
  }
}
```

`provider.getTransactionReceipt(txHash)` has two failure modes that both trigger the vulnerable path:

1. **Network / RPC error:** the JSON-RPC call throws (timeout, connection reset, upstream 5xx, ethers `SERVER_ERROR`). The catch block fires with `transferSucceeded = false`.
2. **Receipt not yet propagated:** the call succeeds but returns `null` because the transaction was just mined and the load-balanced RPC endpoint has not observed the block. The `!receipt` branch throws. The catch block fires with `transferSucceeded = false`.

Both conditions are routine on Polygon: block times are approximately 2 seconds, RPC provider load balancing is common, and `getTransactionReceipt` returning null for a just-mined transaction is normal behavior.



Exploit path (no attacker needed; natural RPC flake suffices):

1. User's `pending_commission = $X`. User requests withdrawal.
2. Worker runs. `resetPendingCommission` zeros the balance and returns `$X`.
3. `sdk.transferUsdc` succeeds. USDC leaves the master Safe and lands in the user's Safe.
4. `provider.getTransactionReceipt(txHash)` throws or returns null.
5. Catch block: `transferSucceeded` is still `false`.
`addPendingCommission(userId, $X)` restores the balance.
6. `referral_commissions` rows stay in `credited` state because `markWithdrawn` was never reached.
7. User requests withdrawal again. Worker sends another `$X`. Repeat while RPC remains flaky.

The `$X` received is real on-chain value; the restored `pending_commission` is a fresh ledger entry that can be withdrawn again. `getTotalEarnings` continues to report the commissions as earned and not withdrawn, matching the restored ledger state.

The worker path fires the sequence; the DB-layer primitives (`addPendingCommission` , `resetPendingCommission` , `markWithdrawn`) fail to treat "USDC transferred" as an authoritative commitment point, which is what permits the double credit.

Affected files

- `packages/queue/src/workers/referral-withdraw.worker.ts:89-171` : worker restores `pendingCommission` on any post-transfer error.
- `packages/database/src/services/users.ts:377-391` : `addPendingCommission` restores balance unconditionally; no idempotency key on (userId, txHash).
- `packages/database/src/services/users.ts:397-419` : `resetPendingCommission` CTE zeros the balance; no withdrawal-attempt ledger.
- `packages/database/src/services/commissions.ts:117-137` : `markWithdrawn` is only reached after `transferSucceeded` is true; it never runs on this failure path.

Impact: High

Repeated withdrawal of the same commission balance. Every RPC flake (or post-transfer fault) between `sdk.transferUsdc` succeeding and `transferSucceeded = true` being assigned causes the user's `pending_commission` to be restored even though the USDC has already left the master Safe. The next withdrawal request pays the same `$X` again.

Direct loss is `pending_commission × (N-1)` where N is the number of withdrawal cycles the user runs before operations notice. There is no protocol-side guardrail: `markWithdrawn` never fires on this path, so `referral_commissions.status` stays `credited` and `getTotalEarnings` reports the commissions as still earnable. No aggregate reconciliation would catch the discrepancy without manual intervention.

Under normal Polygon RPC p99 latencies and occasional provider blips, the condition is hit naturally across the user base.



Precursor dependency: This finding is currently masked by the `resetPendingCommission` wrong-result-shape defect (described as a separate finding) — the referral-withdraw worker is never invoked with a non-zero amount today. The defect becomes exploitable immediately once that precursor is fixed.

Recommendations

Commit the "transfer succeeded" observation BEFORE any post-transfer operation that can throw. Two viable fixes:

1. **Trust the SDK's success signal.** If `sdk.transferUsdc` reports `result.success = true` with a `txHash`, set `transferSucceeded = true` immediately. Perform the receipt check inside its own try/catch that logs but does not restore:

```
typescript if (!result.success) throw new Error('USDC transfer failed: ...'); const txHash = result.transactionHash || ''; if (!txHash) throw new Error('Transfer returned no transaction hash');
```



```
// From this point, the transfer is authoritative. Never restore. transferSucceeded = true;

// Receipt check is observability, not a gate. try { const receipt = await
provider.getTransactionReceipt(txHash); if (!receipt || receipt.status !== 1)
{ logger.warn('Receipt check non-success after SDK success', { txHash }); } } catch (e)
{ logger.warn('Receipt fetch failed; SDK reported success', { txHash, error: e }); }
```
2. **Add a withdrawal-attempt ledger.** A dedicated table `referral_withdrawal_attempts(id, user_id, amount, tx_hash, status)` with `UNIQUE(tx_hash)`. Write a row with `status='submitted'` before calling `transferUsdc`. On SDK success, transition to `on_chain`. Restoring to `pending_commission` only happens for rows still at `submitted` after a bounded retry window, and `addPendingCommission` should refuse (or be scoped) when an attempt row exists with `status IN ('on_chain', 'settled')`.

Defense-in-depth on the DB side: `markWithdrawn` should also be called on the success-through-receipt-check path; alternatively, move the `referral_commissions → withdrawn` state transition immediately after the SDK reports success, before the receipt check. This decouples on-chain observation from ledger commitment.

[H-02] `resetPendingCommission` returns zero instead of user pending commission value

Severity

Impact: High

Likelihood: Medium



Description

`resetPendingCommission` uses raw SQL with a CTE designed to atomically zero `users.pending_commission` and return the pre-update value so the caller can bridge that amount out via USDC. The UPDATE does run; the row is zeroed in Postgres. However, the JS layer reads the returned value through the wrong property, so the function always returns '0'.

```
// packages/database/src/services/users.ts:397-419
async resetPendingCommission(userId: string): Promise<string> {
  try {
    // Use raw SQL to atomically reset and capture the previous value.
    // Drizzle's .returning() returns post-update values, so we use a CTE
    // to read the old value before the update applies.
    const result = await this.db.execute(sql`
      WITH old AS (
        SELECT id, COALESCE(pending_commission, '0') AS prev
        FROM users WHERE id = ${userId} AND COALESCE(pending_commission, '0')::numeric > 0
        FOR UPDATE
      )
      UPDATE users SET pending_commission = '0', updated_at = NOW()
      FROM old WHERE users.id = old.id
      RETURNING old.prev AS previous_commission
    `);

    const previousAmount = (result as any).rows?.[0]?.previous_commission || '0';
    logger.info('Reset pending commission', { userId, previousAmount });
    return previousAmount;
  } catch (error) {
    throw new DatabaseError(`Failed to reset pending commission: ${error instanceof
Error ? error.message : 'Unknown error'}`);
  }
}
```

The repo uses `drizzle-orm@^0.45.1` with the `postgres-js` adapter (see `packages/database/src/client/index.ts`). For that driver, `db.execute(sql\ ...)` returns the postgres-js Result directly: an array-like object with no .rows property. (result as any).rows evaluates to undefined, undefined?.[0]?.previous_commission is undefined, and || '0' gives back the string '0' on every call. The as any` cast is what silences the TypeScript error that would have caught this at compile time.`



Every other `db.execute(sql\ ...)`` call site in this package correctly treats the result as an array. This is the sole outlier:

- `packages/database/src/services/referrals.ts:110` : `const rows = result as unknown as Array<{ level: number; count: string }>;`
- `packages/database/src/services/referrals.ts:146` : `const rows = result as unknown as Array<{ total: string }>;`
- `packages/database/src/services/commissions.ts:107` : `const rows = result as unknown as Array<{ total: string }>;`
- `packages/database/src/services/commissions.ts:152` : `const rows = result as unknown as Array<{ level: number; total: string }>;`
- `packages/database/src/services/temporary-wallets.ts:88-96` : `const rows = result; if (!rows || rows.length === 0) ... const row = rows[0];`

Five independent sites in the same codebase use the array form. Only `users.ts:413` uses `.rows`.

Caller in the bot's referral-withdraw handler:

```
// apps/bot-v2/src/index.ts:1897-1908
try {
  // Atomically reset and capture the pending commission
  const previousCommission = await userService.resetPendingCommission(user.id);
  const pendingCommission = parseFloat(previousCommission);
  if (pendingCommission < REFERRAL_MIN_WITHDRAWAL) {
    // Restore the balance if below minimum (was reset to 0)
    if (pendingCommission > 0) {
      await userService.addPendingCommission(user.id, pendingCommission);
    }
    await ctx.reply(`⚠ Minimum withdrawal amount is ${REFERRAL_MIN_WITHDRAWAL} USDC. Your
balance: ${pendingCommission.toFixed(2)}`);
    return;
  }
}
```

The sequence on any user tapping "Withdraw Earnings" with any non-zero pending balance:

1. DB UPDATE fires; `pending_commission` goes from `$X` to `'0'`.
2. Function returns `'0'`.
3. Caller parses `'0'` → `pendingCommission = 0`.
4. `0 < REFERRAL_MIN_WITHDRAWAL` → enters the "below minimum" branch.
5. Restore guard `if (pendingCommission > 0)` is false (`0 > 0`), so `addPendingCommission` is NOT called.
6. User sees "Minimum withdrawal amount is \$5 USDC. Your balance: \$0.00" and is returned out.
7. Even users whose balance was above the minimum are affected: the enqueued job is given `amount: 0`, the worker does nothing useful, and the `referral_commissions` rows are never flipped to `withdrawn`.



The cast `as any` is precisely the type-safety escape hatch that hides this. Without it, `tsc` would have refused the `.rows` access.

Affected files

- `packages/database/src/services/users.ts:397-419` : `resetPendingCommission` 's `.rows?.[0]?.previous_commission` is always `undefined`, so the function always returns `'0'`.
- `apps/bot-v2/src/index.ts:1897-1908` : caller trusts the return value as the pre-reset balance; restore guard (`pendingCommission > 0`) is false on `0`, so the wipe is never reversed.
- `packages/database/src/services/referrals.ts:110,146` : correct array shape (`rows[0]?.total`).
- `packages/database/src/services/commissions.ts:107,152` : correct array shape.
- `packages/database/src/services/temporary-wallets.ts:88-96` : correct array shape (`const rows = result; rows.length; rows[0]`).

Impact: High

Every user with a non-zero `pending_commission` who taps "Withdraw Earnings" in the bot has their full accrued commission balance silently zeroed (recoverable only via DBA reconciliation of the referral_commissions ledger) without a withdrawal being dispatched. The UPDATE writes `pending_commission = '0'` to the DB; the caller thinks the user's balance was always zero; no `addPendingCommission` restore runs; no queue job is meaningfully enqueued. The `referral_commissions` ledger rows remain in `status='credited'` forever, diverging from the `users.pending_commission` column, which states zero. From the user's perspective, money accrued over days or weeks disappears the first time they try to withdraw.

This is deterministic, reproducible on every call, and hits every user of the referral-withdrawal flow. Recovery requires DBA reconciliation between `referral_commissions` aggregates and `users.pending_commission`. **Impact:** High (quantifiable off-chain fund loss, reconciliation recoverable), **Likelihood:** High (any user click triggers it), **Severity:** High.

Recommendations

Replace line 413 with the array form that the rest of the codebase uses:

```
const rows = result as unknown as Array<{ previous_commission: string }>;
const previousAmount = rows[0]?.previous_commission ?? '0';
```



Additionally:

1. Remove the `as any` cast. There is no legitimate reason to widen the `execute()` return type to `any`. Use `as unknown as Array<...>` the way every other service in this package does. The cast is hiding a bug class, not enabling a feature.
2. Add an integration test that writes a non-zero `pending_commission`, calls `resetPendingCommission`, and asserts the RETURN value equals the previously written amount (not just that the DB column is zero).
3. A lint rule forbidding `.rows` on `db.execute()` return values prevents the regression class.



Medium findings

[M-01] Fee transfer dedup relies on migration only unique index not in schema

Severity

Impact: Medium

Likelihood: Medium

Description

`FeeTransferService.create` uses `onConflictDoNothing` for deduplication, but the intended `trade_id` plus `telegram_id` uniqueness is only defined in a later SQL migration and is not represented in the `Drizzle` schema. The migrations journal also stops at earlier migration tags. If the deployment state misses that unique index, retries can insert duplicate pending fee rows and trigger repeated fee transfer attempts for the same trade.

Recommendations

Mirror the partial unique index in `schema.ts`, repair migration journal tracking for all migration files, and use an explicit `onConflictDoNothing` target on `trade_id` and `telegram_id`. Add a startup assertion that the expected deduplication index exists before starting fee workers.

[M-02] Promo can be stuck in `processing` state due to missing revert logic

Severity

Impact: High

Likelihood: Low

Description

In the promo claim flow, the `KolPromoService::markPromoProcessing()` function is used to transition a promo to the `processing` state before executing the claim. However, if any subsequent step fails or exits early (e.g., Telegram `replyWithAnimation` failure, Redis unavailability, or `promoClaimQueue.add` throwing), the `catch` block only sends an error response and returns. It does not call `KolPromoService::revertPromoToPending()`.



As a result, the promo remains permanently stuck in the `processing` state. Since future claim attempts typically require the promo to be in the `pending` state, the user is unable to retry, effectively causing a permanent denial of service for that promo.

Impacts

Affected users are unable to claim their promo rewards, as the promo becomes permanently locked in the `processing` state.

Recommendations

- Move all pre-checks (e.g., user validation, config validation) before calling `markPromoProcessing()`.
- In the `catch` block, explicitly call `KolPromoService::revertPromoToPending()` if the promo has already been transitioned to `processing`.

```
try {
  const user = await ctx.ensureUser();
  if (!user.gnosisSafeAddress) {
    await ctx.reply('Please create a wallet first.');
```

`return;`

```
  }

  const config = getConfig();
  if (!config.REDIS_URL) {
    await ctx.reply('❌ Promo claim service unavailable. Please try again later.');
```

`return;`

```
  }

  const userPromoDetails = await ctx.kolPromoService.getUserPromoWithDetails(user.id);
  if (!userPromoDetails || userPromoDetails.userPromo.status !== 'pending') {
    await ctx.reply('No pending promo found or already claimed.');
```

`return;`

```
  }
  ...
} catch (error) {
  logger.error('Failed to process promo claim', { error });
  // Call to `KolPromoService::revertPromoToPending` function
  await ctx.reply('❌ Failed to process promo claim. Please try again.');
```

`}`

As a defense-in-depth measure, add a `processing_started_at` timestamp column to the promo table and implement a background sweeper job that reverts any promo stuck in `processing` state for longer than a configurable timeout (e.g., 5 minutes) back to `pending`, enabling automatic liveness recovery without manual operator intervention.



[M-03] `revertPromoToPending` allows reversion after successful onchain transfer

Severity

Impact: Medium

Likelihood: Medium

Description

`revertPromoToPending` is the DB-layer primitive that the promo-claim worker calls from its outer `catch` block to put a promo row back into `pending` status when anything goes wrong. It filters only on the current status being `processing`, with no check that a successful on-chain transfer has already occurred:

```
// packages/database/src/services/kol-promo.ts:155-167
async revertPromoToPending(userPromoId: string): Promise<boolean> {
  try {
    const result = await this.db.update(userPromos)
      .set({ status: 'pending', updatedAt: new Date() })
      .where(and(eq(userPromos.id, userPromoId), eq(userPromos.status, 'processing')))
      .returning({ id: userPromos.id });

    return result.length > 0;
  } catch (error) {
    logger.error({ error, userPromoId }, 'Failed to revert promo to pending');
    return false;
  }
}
```

The sister `claimPromo` is the function that would mark the successful state transition. It writes `status='claimed'`, `claimTxHash`, `claimedAmount`, `claimedAt`, `unlockAt`:

```
// packages/database/src/services/kol-promo.ts:172-223
async claimPromo(userPromoId: string, txHash: string, amount: string): Promise<UserPromo> {
  try {
    // Validate txHash is a non-empty, valid hex transaction hash
    if (!txHash || !/^0x[a-fA-F0-9]{64}$/.test(txHash)) {
      throw new DatabaseError(`Invalid transaction hash: ${txHash} || '(empty)'`);
    }

    // Look up the associated userPromo to get kolPromoId
    const userPromo = await this.db.query.userPromos.findFirst({
      where: (up, { eq }) => eq(up.id, userPromoId),
    });
    if (!userPromo) {
      throw new DatabaseError('User promo not found');
    }

    // Validate amount matches the KOL promo's claimableAmount
    const kolPromo = await this.db.query.kolPromos.findFirst({
```



```
    where: (kp, { eq }) => eq(kp.id, userPromo.kolPromoId),
  });
  if (!kolPromo) {
    throw new DatabaseError('Associated KOL promo not found');
  }
  if (amount !== kolPromo.claimableAmount) {
    throw new DatabaseError(`Claim amount ${amount} does not match promo claimable
amount ${kolPromo.claimableAmount}`);
  }
  // ... continues with the UPDATE setting status='claimed', claimTxHash, etc.
```

The user-re-click exploit path is gated by BullMQ idempotency: the claim queue uses `jobId: promo-claim-${userPromoId}` with `removeOnFail: 500`, so once a job enters the queue a user cannot simply re-click "Claim" in the UI to fire a second job with the same id until the first is evicted. The real triggers for the double-spend chain are two:

1. **BullMQ retry if `attempts` is raised.** Current queue config is `attempts: 1` (trading.queue.ts:58), so the retry loop is not a real current path. If `attempts` is ever raised above 1 (the comment in the queue config suggests this was intended), each retry re-invokes `sdk.transferUsdc` without checking whether a successful transfer already occurred; the first retry after a successful transfer is a direct treasury double-spend.
2. **Admin edit of `claimableAmount` between enqueue and execute.** Admin edits `claimableAmount` on an active promo between user click and worker execution. USDC is transferred at the old amount; `claimPromo` throws on amount mismatch; `revertPromoToPending` flips status back to `pending`. Under `attempts: 1`, the failed BullMQ job sits in the `removeOnFail: 500` buffer with the same `jobId`, deduping user re-clicks until the buffer evicts. Net impact: 1x treasury loss with no DB claim record; potential replay after buffer eviction (500 subsequent claim failures).

The DB-layer primitive `revertPromoToPending` is the unsafe enabler: the worker throws BEFORE the `claim_tx_hash` UPDATE runs (the amount-mismatch guard fires at the guard step, not after the UPDATE), so `claim_tx_hash` is never populated. An `isNull(claim_tx_hash)` predicate on the revert would not block this path at all.

Affected files

- `packages/database/src/services/kol-promo.ts:155-167` : `revertPromoToPending` filters only on `status='processing'`; no `claim_tx_hash` check could help because the hash is never written on the throwing paths.
- `packages/database/src/services/kol-promo.ts:172-223` : `claimPromo` is the source of the post-transfer failure modes. The amount-mismatch guard at line 194 fires after on-chain success but before any DB write that would record the transfer.

Impact: Medium

Under current queue config (`attempts: 1` , `removeOnFail: 500`), the admin-edit trigger causes a single-transfer treasury loss with no user-side claim record: the master Safe disburses `claimableAmount` USDC, `claimPromo` throws on amount mismatch, `revertPromoToPending` flips the row back to `pending`, and the failed BullMQ job sits in the



`removeOnFail: 500` buffer deduping re-clicks until eviction. A replay cycle requires either buffer eviction (500 subsequent claim failures) or `attempts` being raised above 1; at that point, each retry becomes a direct treasury double spend against the same promo. The bug requires either (a) an admin who edits `claimableAmount` on an active promo while users are mid-claim, or (b) a future change that enables worker retries. Neither requires attacker action, but neither is an everyday occurrence either, which caps likelihood at Medium.

Recommendations

The correct fix is an intermediate `claimed_unconfirmed` status on the promo state machine. Once `sdk.transferUsdc` returns success AND the receipt is verified, the worker atomically writes `status='claimed_unconfirmed'` with `claim_tx_hash` populated, BEFORE running any other fallible step (amount-mismatch guard, notification, logging). Then:

1. `revertPromoToPending` gates on `status='processing'` only, which is correct given that by construction `claim_tx_hash` is never written in `processing` state.
2. The `claimed_unconfirmed` state is terminal for the monetary machine: it cannot revert to `pending` under any path. A separate reconciliation process can promote `claimed_unconfirmed` to `claimed` once all post-transfer bookkeeping succeeds, or mark it for manual review.
3. The BullMQ retry loop becomes safe because the worker's first idempotency check is "is this row already in `claimed_unconfirmed` or `claimed` ? If yes, do not call `sdk.transferUsdc` again."

Additionally:

1. Add a DB CHECK `claim_tx_hash IS NULL OR status IN ('claimed', 'claimed_unconfirmed', 'unlocked')` to structurally forbid a populated-hash-plus-pending combination if any future code path writes the hash early.
2. Store an idempotency key per `(userPromoId, attempt_id)` in a dedicated `promo_claim_attempts` table, and require `sdk.transferUsdc` to check for an existing successful attempt before submitting.
3. Reject `kolPromos.claimableAmount` edits when any `user_promos` row for that promo is in `pending` or `processing` state, to cut off the admin edit trigger.

[M-04] Migrations hold ACCESS EXCLUSIVE on large tables without timeout or concurrency settings

Severity

Impact: Medium

Likelihood: Medium



Description

Several migrations in `packages/database/migrations/` use patterns that hold `ACCESS EXCLUSIVE` on the target table for the duration of a full-table scan or heap rewrite. These patterns are safe on empty test databases and unsafe against production volumes. The migrations have no `SET lock_timeout`, no `SET statement_timeout`, no `CONCURRENTLY` on index builds, and no `NOT VALID` + `VALIDATE CONSTRAINT` split for constraint additions.

0040: 36 ALTER COLUMN SET DATA TYPE with USING Clause

```
-- packages/database/migrations/0040_standardize_timestamps_to_timestamptz.sql:1-20
-- Standardize all timestamp columns to timestamptz (timestamp with time zone)
-- Existing data is assumed UTC; the USING clause preserves values.

ALTER TABLE "users" ALTER COLUMN "created_at" SET DATA TYPE timestamptz USING "created_at"
AT TIME ZONE 'UTC';

--> statement-breakpoint
ALTER TABLE "users" ALTER COLUMN "updated_at" SET DATA TYPE timestamptz USING "updated_at"
AT TIME ZONE 'UTC';
...
ALTER TABLE "market_cached" ALTER COLUMN "created_at" SET DATA TYPE timestamptz USING
"created_at" AT TIME ZONE 'UTC';
```

The full file contains 36 such `ALTER COLUMN ... SET DATA TYPE ... USING ...` statements across all dated tables including `copy_trade_history` (the v1 table whose 50M+ row size is documented in the `0025` migration comment) and the newer `copy_trade_history_2`. `SET DATA TYPE` with a `USING` expression forces a full heap rewrite under `ACCESS EXCLUSIVE`. For the 50M+ row `copy_trade_history` v1 table, this is minutes to hours of lock time during which no read or write succeeds on the table.

0037: 8 CHECK constraints without NOT VALID

```
-- packages/database/migrations/0037_add_check_constraints_status_enums.sql:3-25
ALTER TABLE "users" ADD CONSTRAINT "chk_users_trading_mode"
CHECK ("trading_mode" IN ('cautious', 'standard', 'expert'));

--> statement-breakpoint
ALTER TABLE "referral_commissions" ADD CONSTRAINT "chk_referral_commissions_status"
CHECK ("status" IN ('pending', 'credited', 'withdrawn'));
...
ALTER TABLE "copy_trading_follows" ADD CONSTRAINT "chk_copy_trading_follows_mode"
CHECK ("mode" IN ('proportional', 'fixed', 'percentage'));

--> statement-breakpoint
ALTER TABLE "fee_transfers" ADD CONSTRAINT "chk_fee_transfers_status"
CHECK ("status" IN ('pending', 'success', 'failed'));
```

None of the 8 `ADD CONSTRAINT ... CHECK` statements include `NOT VALID`. Each forces a full-table scan while holding `ACCESS EXCLUSIVE`. For `copy_trading_follows` (potentially millions of rows) and `referral_commissions`, this blocks writes during the scan. The correct



pattern is `ADD CONSTRAINT ... CHECK (...) NOT VALID;` in one transaction, followed by `VALIDATE CONSTRAINT` in a separate transaction (which uses `SHARE UPDATE EXCLUSIVE`, non-blocking for writes).

0034 / 0035 / 0036: UNIQUE constraints without CONCURRENTLY + USING INDEX

```
-- packages/database/migrations/0034_wallets_telegram_id_unique.sql:1
ALTER TABLE "wallets" ADD CONSTRAINT "wallets_telegram_id_unique" UNIQUE ("telegram_id");
```

```
-- packages/database/migrations/0035_users_referral_code_unique.sql:1
ALTER TABLE "users" ADD CONSTRAINT "users_referral_code_unique" UNIQUE ("referral_code");
```

```
-- packages/database/migrations/0036_temporary_wallets_unique_safe_address.sql:1
ALTER TABLE "temporary_wallets" ADD CONSTRAINT
"temporary_wallets_gnosis_safe_address_unique" UNIQUE ("gnosis_safe_address");
```

`ALTER TABLE ... ADD CONSTRAINT ... UNIQUE(col)` takes `ACCESS EXCLUSIVE` for the full index build. On `wallets` (one row per user, at least 10,000 users per the `0027` comment), this write-blocks authentication and wallet lookup for seconds during deployment. The correct pattern is:

```
CREATE UNIQUE INDEX CONCURRENTLY "wallets_telegram_id_unique" ON "wallets" ("telegram_id");
ALTER TABLE "wallets" ADD CONSTRAINT "wallets_telegram_id_unique" USING INDEX
"wallets_telegram_id_unique";
```

`CREATE INDEX CONCURRENTLY` takes `SHARE UPDATE EXCLUSIVE` (does not block writes), and `ADD CONSTRAINT ... USING INDEX` is O(1) because the index already exists.

Affected files

- `packages/database/migrations/0040_standardize_timestamps_to_timestamptz.sql:1-74` : 36 full-table heap rewrites under `ACCESS EXCLUSIVE`, including the v1 `copy_trade_history` (50M+ rows per the `0025` comment).
- `packages/database/migrations/0037_add_check_constraints_status_enums.sql:3-25` : 8 `CHECK` adds without `NOT VALID`.
- `packages/database/migrations/0034_wallets_telegram_id_unique.sql:1` : `UNIQUE` add without `CONCURRENTLY` + `USING INDEX`.
- `packages/database/migrations/0035_users_referral_code_unique.sql:1` : same pattern.
- `packages/database/migrations/0036_temporary_wallets_unique_safe_address.sql:1` : same pattern.



Impact: Medium

Deploy-time write-blocking outage proportional to table size. The duration scales with row count, not with schema complexity, so the pattern is a latent bomb that worsens every day the tables grow. For the v1 `copy_trade_history` table (50M+ rows per the `0025` migration comment), running `0040` against production requires a dedicated maintenance window; the migration as written holds the table locked for the entire heap rewrite.

Drizzle-kit runs migrations sequentially at deploy. If any operator runs `drizzle-kit migrate` against production without first manually gating `0040`, the deploy stalls and the copy-trade pipeline (WebSocket listener, BullMQ workers, user UI settings) all block behind the lock. BullMQ retry policies compound the problem: jobs that fail their DB writes retry, queue up, and saturate Redis while the lock is held.

None of the migrations carry a comment warning operators about the locking behavior, and none include the guard patterns (`SET lock_timeout = '5s'` , `SET statement_timeout = '10min'`) that would at least cause the migration to abort cleanly instead of stalling indefinitely.

Recommendations

For each migration class, adopt the non-blocking pattern:

0040 pattern (column type change):

```
-- Add new column, backfill in batches, rename, drop old.
-- OR if the rewrite is required, gate it with:
SET lock_timeout = '5s';
SET statement_timeout = '30min';
ALTER TABLE "users" ALTER COLUMN "created_at" SET DATA TYPE timestamptz
    USING "created_at" AT TIME ZONE 'UTC';
```

If any table exceeds ~1M rows, do the add-new-column + batched-backfill + rename approach instead of `SET DATA TYPE`.

0037 pattern (CHECK constraints):

```
ALTER TABLE "users" ADD CONSTRAINT "chk_users_trading_mode"
    CHECK ("trading_mode" IN ('cautious', 'standard', 'expert')) NOT VALID;

-- separate transaction:
ALTER TABLE "users" VALIDATE CONSTRAINT "chk_users_trading_mode";
```

0034/0035/0036 pattern (UNIQUE constraints):

```
CREATE UNIQUE INDEX CONCURRENTLY "wallets_telegram_id_unique"
    ON "wallets" ("telegram_id");
ALTER TABLE "wallets" ADD CONSTRAINT "wallets_telegram_id_unique"
    USING INDEX "wallets_telegram_id_unique";
```



Going forward, adopt a project convention that every migration explicitly sets `lock_timeout` and `statement_timeout` at the top. A CI lint step that rejects `ALTER TABLE ... ADD CONSTRAINT UNIQUE`, `ADD CONSTRAINT CHECK` without `NOT VALID`, and `SET DATA TYPE` on tables above a configured row count threshold would catch this class before merging.

[M-05] `updateFill` lacks idempotency guard leading to inflated `filledSize` in orders

Severity

Impact: Medium

Likelihood: Medium

Description

`updateFill` is the DB primitive the scanner calls each time it observes an `OrderFilled` event for a limit order. It increments `filledSize` via a raw SQL `COALESCE(...) + fillAmount`, gated solely on the order's primary key:

```
// packages/database/src/services/limit-orders.ts:163-189
async updateFill(id: string, fillAmount: number): Promise<{ newFilledSize: number;
originalSize: number }> {
  try {
    const [updated] = await this.db.update(limitOrders)
      .set({
        filledSize: sql`COALESCE(${limitOrders.filledSize}, 0) + ${fillAmount}`,
        updatedAt: new Date(),
      })
      .where(eq(limitOrders.id, id))
      .returning({
        filledSize: limitOrders.filledSize,
        originalSize: limitOrders.originalSize,
      });

    if (!updated) {
      throw new DatabaseError('Limit order not found');
    }

    const newFilledSize = parseFloat(updated.filledSize ?? '0');
    const originalSize = parseFloat(updated.originalSize ?? '0');

    logger.info('Limit order fill updated', { id, fillAmount, newFilledSize });
    return { newFilledSize, originalSize };
  } catch (error) {
    if (error instanceof DatabaseError) throw error;
    throw new DatabaseError(`Failed to update fill: ${error instanceof Error ?
error.message : 'Unknown error'}`);
  }
}
```




None of the following defenses exist:

1. No `isTakenFee = false` predicate in the WHERE clause. The SQL increments unconditionally even if the order is already marked as fully settled and fee taken.
2. No `isFullyFilled = false` predicate. Once an order has crossed the downstream `markFullyFilled` threshold, a re-delivered event still adds to `filledSize`.
3. No event deduplication table. There is no `(order_hash, block_number, log_index)` junction table consulted before inserting. No record of which chain events have been processed.
4. No cap. `filledSize` is allowed to exceed `originalSize`; the SQL has no `LEAST(..., originalSize)` or `filled_size + fill_amount <= original_size` CHECK.
5. No optimistic concurrency token. The caller does not pass an expected prior `filledSize` for CAS-style updates.

The service comment explicitly claims, "Uses SQL increment to avoid race conditions between concurrent fills." That guards against two concurrent writers stomping each other's additions, but it does not guard against replay of the same underlying chain event. A replay of the same `OrderFilled` log (which happens when the scanner re-scans a block range, when two scanner instances overlap, or when a job retries after a transient DB error) drives `filledSize` upward every time. Once `filledSize / originalSize >= 0.99` (the downstream threshold used by the scanner's fee-processing path), the order is flipped to `isFullyFilled=true`, and a fee-processing job is queued, regardless of whether the on-chain position is actually 99% filled. A 50%-filled order re-processed twice crosses the threshold.

The return signature parses back through `parseFloat`, which loses precision on large decimals. Not the primary concern, but it compounds the inflation: accumulated float error grows with each repeated addition, and downstream comparisons use the float-rounded value.

Affected files

- `packages/database/src/services/limit-orders.ts:163-189`: `updateFill` uses an unconditional SQL increment, no idempotency key, no cap, and no source-state guard.

Impact: High

The protocol collects fees on what it believes are fully filled limit orders but are economically partial fills. Every legitimate partial fill event that gets re-delivered by the scanner drives `filledSize` upward until it crosses the 99% threshold, at which point `markFullyFilled` flips the order and the downstream fee processing worker fires for the full `feeAmount`. The Treasury collects fees that exceed actual on-chain fills.

User-visible effect is also wrong: the `order-fill-notify` pipeline messages the user claiming the order is fully filled when on-chain it is still partial. The user sees a "your order filled" notification and acts on that belief even though the on-chain position is half executed.



Likelihood is Medium because the scanner's backward walk and re-sweep behavior is what triggers it. The DB primitive has zero defense against it. In a rolling deploy overlap where two scanners run briefly, every limit order with a fill in the overlap block gets double-counted.

Recommendations

Pick one of three correct designs. All require changes to this primitive:

1. **Event-dedup table (preferred).** Introduce `limit_order_fills` with `UNIQUE(order_hash, block_number, log_index)`. The scanner inserts one row per event via `.onConflictDoNothing()`. The service computes `filledSize` as `SUM(fill_events.amount) WHERE order_id = ?` from that table. Replays are idempotent by construction.
2. **Cap + source-state guard on the current primitive.** Add predicates and a cap:

```
typescript .set({ filledSize: sql`LEAST( COALESCE($
{limitOrders.filledSize}, 0) + ${fillAmount}, $
{limitOrders.originalSize} )`, updatedAt: new
Date(), }) .where(and( eq(limitOrders.id, id), eq(limitOrders.isTakenFee,
false), eq(limitOrders.isFullyFilled, false), ))
```

 Also add a DB-level CHECK:
`filled_size <= original_size`.
3. **CAS-style caller contract.** Change the signature to `updateFill(id, expectedFilledSize, fillAmount)` and gate on `eq(limitOrders.filledSize, expectedFilledSize)`. Caller handles mismatch by re-reading.

Additionally, `parseFloat` on `decimal(18,6)` columns should be replaced with string-based comparison or a big-decimal library for the threshold check, so the 99% decision does not depend on float rounding.

[M-06] Missing unique constraint on `fee_transfers` leads to ineffective deduplication

Severity

Impact: Medium

Likelihood: Medium

Description

`FeeTransferService.create()` at `packages/database/src/services/fee-transfers.ts:65-84` attempts to deduplicate fee-transfer records with `.onConflictDoNothing()`:

```
const [record] = await this.db.insert(feeTransfers).values({
  telegramId: params.telegramId,
  tradeType: params.tradeType,
  tradeId: params.tradeId,
```



```
...
status: 'pending',
}).onConflictDoNothing().returning();

if (!record) {
  logger.info('Fee transfer already exists (duplicate trade), skipped', { ... });
  return null;
}
```

In `schema.ts`, the `fee_transfers` table has no unique constraint on `tradeId` (or any combination that would uniquely identify a logical fee event). The only unique target is the `serial` primary key, which is auto-generated and can never collide. `ON CONFLICT DO NOTHING` without an explicit target applies to any unique/PK constraint — since none exists on `tradeId`, the clause is dead code. Every call unconditionally inserts a new row, and the `record` branch is unreachable.

If the fee-transfer worker (or any upstream buy/sell/claim/copy-trade worker) is redelivered by BullMQ after a crash, timeout, or manual retry, `create()` inserts a second pending row with the same `tradeId`. The worker then executes a second on-chain USDC transfer from the user's Safe to the master fee address. The user is charged the platform fee twice. This directly contradicts the `COMMISSION_FIX.md` "Retry Case" invariant ("Fee Transfer Fails (retry) → Eventually Succeeds → Commission Recorded Once").

Recommendations

Add a partial unique constraint that expresses the real business key, and specify the conflict target explicitly:

```
CREATE UNIQUE INDEX idx_fee_transfers_trade_dedup
ON fee_transfers (telegram_id, trade_type, trade_id)
WHERE trade_id IS NOT NULL;
```

```
.onConflictDoNothing({ target: [feeTransfers.telegramId, feeTransfers.tradeType,
feeTransfers.tradeId] })
```

Additionally, wrap the create → on-chain submit → markSuccess sequence in a structure that is idempotent by `tradeId` (e.g. look up an existing `status = success` row with the same `tradeId` before submitting a new transfer).

[M-07] Referral percent override cannot be applied under the current user creation flow

Severity

Impact: Medium

Likelihood: Medium



Description

The application's user-creation flow is INSERT-then-UPDATE on `referred_by_id` :

```
// users.ts:126-131 (inside findOrCreateByTelegramId)
const [newUser] = await this.db.insert(users).values({
  telegramId,
  username: username ?? null,
  firstName: firstName ?? null,
  lastName: lastName ?? null,
}).returning();
```

`referred_by_id` is never supplied at INSERT. It defaults to NULL. The referrer is attached later by `setReferrer` :

```
// users.ts:343-372
async setReferrer(userId: string, referrerId: string): Promise<void> {
  // ...
  const newLevel = Math.min((referrer.referralLevel || 0) + 1, 3);

  const result = await this.db.update(users)
    .set({
      referredById: referrerId,
      referralLevel: newLevel,
      updatedAt: new Date(),
    })
    .where(and(eq(users.id, userId), isNull(users.referredById)))
    .returning({ id: users.id });
  // ...
}
```

`setReferrer` only writes `referredById` and `referralLevel` . It does not recompute `referralPercents` . The user keeps the schema default forever:

```
// schema.ts:19
referralPercents: text('referral_percents').default('[0.25, 0.05, 0.03]'), // [F1%, F2%, F3%]
```

A BEFORE-INSERT trigger in `packages/database/migrations/0017_referral_percent_trigger.sql` is designed to override `NEW.referral_percents` when `NEW.referred_by_id` matches a VIP UUID. That trigger can never fire in the application flow because `referred_by_id` is always NULL at INSERT time. The override logic is dead.

Affected files

- `packages/database/src/services/users.ts:126-131` : `findOrCreateByTelegramId` inserts with `referred_by_id = NULL` .
- `packages/database/src/services/users.ts:343-372` : `setReferrer` updates `referred_by_id` later but never rewrites `referral_percents` .
- `packages/database/src/schema.ts:19` : `referralPercents` defaults to the non-VIP rate.



Impact: High

When U is referred by VIP, the trigger's intent is for U's own `referral_percents` to become `[0.15, 0.05, 0.03]`. Because the trigger can never fire (the app INSERTs with `referred_by_id = NULL`), U's column stays at the schema default `[0.25, 0.05, 0.03]`. The effect surfaces only when U themselves become a referrer: every time one of U's own referred users (V) trades, U's Level-1 commission is paid at 25% of the fee instead of the intended 15%. The 66.67% overpayment on every V trade is charged to the protocol's commission accrual on every trade by V (and V's downline). The beneficiary is U (a VIP's direct downline), not VIP themselves. The error is silent; there is no reconciliation path. Historical overpayments are not recoverable; only future trades can be corrected once the fix is in place.

Recommendations

Fix inside `setReferrer` (in scope), not in the trigger:

```
// users.ts: inside setReferrer
const VIP_REFERRER_IDS = new Set<string>(['<vip-uuid-1>', '<vip-uuid-2>']); // from config
const isVIP = VIP_REFERRER_IDS.has(referrerId);
const vipPercents = '[0.15, 0.05, 0.03]';

const updateData: Record<string, unknown> = {
  referredById: referrerId,
  referralLevel: newLevel,
  updatedAt: new Date(),
};
if (isVIP) {
  updateData.referralPercents = vipPercents;
}

await this.db.update(users).set(updateData).where(...);
```

Better long-term approach: introduce a `referral_tier_overrides` table keyed on `beneficiary_id` with the override percents. `setReferrer` consults this table and applies the override. Adding a new VIP is a DB insert, not a code change.

After deploying the fix, run the existing `update-existing-users-referral.sql` backfill to correct historical rows.



Low findings

[L-01] Referral chain lookups allow banned users to accrue commissions

Description

`Referral.md` describes the referral program as a commission-bearing feature, and `schema.ts:29` adds `isBanned` with the comment "Admin can ban exploiting users". Only `UserService.getAllUsersWithWallets` enforces `eq(users.isBanned, false)`. `findByReferralCode` (used during new-user onboarding to link a referrer), `getDirectReferrals`, and `ReferralService.getReferralChain` (used when recording commissions) do not exclude banned users. A banned referrer still accrues F1/F2/F3 commissions and can still be linked to by new users — banning is not enforced on the commission accrual path. Recommendation: add `eq(users.isBanned, false)` to these read paths, or strip banned users out of `getReferralChain` before returning the chain.

[L-02] `users.telegramId` lacks unique index in `schema.ts` risking data integrity

Description

`packages/database/src/schema.ts` declares `telegramId: text('telegram_id').notNull()` — no `.unique()` — while the deployed DB has the partial unique index `idx_users_telegram_id_active` from migration 0039. If anyone runs `drizzle-kit push` (direct schema sync), Drizzle diffs source-of-truth `schema.ts` against the live DB and will propose to drop the partial unique index. Even if `drizzle-kit generate` is used, future auto-generated migrations may not recreate the partial unique if the source schema does not express it. Recommendation: represent the partial unique in `schema.ts` using a `uniqueIndex('idx_users_telegram_id_active').on(table.telegramId).where(sql + " ${table.deletedAt} IS NULL " + )` inside the second-argument config object. This pins the ORM-side understanding to the DB-side reality.

[L-03] Missing state update leaves wallet creation flag incorrect

Description

Description



`UserService::updateWalletCreated()` is the only method that writes to the `created_wallet` column, but it is never invoked anywhere in the codebase - meaning every user's `createdWallet` flag stays at its schema default of `false` indefinitely, even after a wallet is fully assigned.

Recommendation

Update the `UserService::updateWalletInfo()` function to also set `createdWallet = true` once the wallet has been successfully created and assigned.

[L-04] `updateLastScannedBlock` overwrites block number causing concurrency issues

Description

`updateLastScannedBlock` overwrites the stored block number unconditionally. There are no SET-if-greater semantics, no `FOR UPDATE` lock, no version or timestamp check, and no leader-election guard at the caller layer:

```
// packages/database/src/services/scanner-state.ts:74-84
async updateLastScannedBlock(contractAddress: string, blockNumber: number): Promise<void> {
  try {
    await this.db.update(scannerState)
      .set({ lastScannedBlock: blockNumber, updatedAt: new Date() })
      .where(eq(scannerState.contractAddress, contractAddress));

    logger.debug('Scanner state updated', { contractAddress, lastScannedBlock:
      blockNumber });
  } catch (error) {
    throw new DatabaseError(`Failed to update last scanned block: ${error instanceof
      Error ? error.message : 'Unknown error'}`);
  }
}
```

Any caller with any value can overwrite the checkpoint. The service provides no way to express "only advance if the new value is strictly greater" or "only write if my expected previous value still holds." Two scanner instances that share the same `contractAddress` (a rolling-deploy overlap, an accidental double start, or a horizontal scaling configuration that nobody remembered to make leader-elected) produce non-deterministic writes.

The situation is worse in this codebase than for a simple forward-scanning watcher because the scanner walks backward. `apps/scanner/src/scanner/base-scanner.ts` decrements `lastScannedBlock` down to 0 and then snaps back to the current head, then decrements again. Two scanners racing on the same state produce arbitrary orderings: scanner-A writes `lastScanned=1000`, scanner-B writes `lastScanned=990`, scanner-A on the next iteration reads 990 (thought it wrote 1000), uses `fromBlock=980, toBlock=990`, writes 980. The range 990..1000 was never processed by either instance; it fell into the gap between the two writes. Any `OrderFilled` event in that gap is silently dropped: no fee collected, no user notification sent.



The same pattern repeats indefinitely while both scanners are live. Every few iterations, small gaps appear; some contain events, some do not; the protocol has no way to know.

This compounds with the unbounded `updateFill` re-sweep: when two scanners do overlap on a range, they both call `updateFill` for the same `(orderHash, logIndex)`, inflating `filledSize` twice. One scanner's write races with the other; both succeed independently; the order's `filledSize` advances by `2 × fillAmount` for a single real fill.

Exploit path (operational, not attacker-driven):

1. The operator runs a rolling deploy of the scanner: the new pod starts while the old one is still draining. For the overlap window (~30 seconds under a typical K8s rollingUpdate configuration), two scanner instances are both polling and both writing `lastScannedBlock`.
2. Both read the same state. Both compute the same `fromBlock` / `toBlock`. Both process the same events. Both write back independently.
3. Under adverse scheduling, the write ordering creates gaps where neither scanner owns a specific block range.
4. A user's limit-order fill fires in the gap. Neither scanner processes it. The order stays `isTakenFee=false, isFullyFilled=false` despite on-chain completion. No fee is collected; no user notification is sent.
5. Eventually, the backward walk of one scanner sweeps past the gap and re-processes the event; but by then, compounded with the `updateFill` idempotency gap, the fill may land double.

Affected files

- `packages/database/src/services/scanner-state.ts:74-84` : `updateLastScannedBlock` unconditional SET.
- `apps/scanner/src/scanner/base-scanner.ts` : caller decrements `lastScannedBlock` each iteration; no single-writer enforcement.

Impact: Medium

Missed OrderFilled events translate directly to missed fees (lost protocol revenue per order) and missed user-facing notifications ("your limit order filled"). In an overlap window of 30 seconds during a rolling deploy with moderate order flow, the number of affected orders is small per incident, but the failure is silent: there is no log, no metric, and no reconciliation that would surface it.

Compounded with the `updateFill` idempotency gap, the double-scanner condition also causes `filledSize` to advance past the true on-chain value. Orders cross the 99% threshold prematurely, trigger the fee-transfer flow for the full fee on a partially filled order, and send the user a "fully filled" notification that is materially wrong.



Likelihood is low under normal single-instance deployment, but the service contract permits multi-writer use without any safeguards, and common operational scenarios (rolling deploy, accidental double-start, horizontal scaling) exercise the bug silently.

Recommendations

Change the update to SET-if-different-by-expected-ordering. Two viable patterns:

1. **Compare-and-set with expected-previous:** `typescript async`
`updateLastScannedBlock(contractAddress: string, expectedPrevious: number, blockNumber: number,): Promise<boolean> { const result = await this.db.update(scannerState) .set({ lastScannedBlock: blockNumber, updatedAt: new Date() }) .where(and(eq(scannerState.contractAddress, contractAddress), eq(scannerState.lastScannedBlock, expectedPrevious),)) .returning({ id: scannerState.contractAddress }); return result.length > 0; // false = concurrent writer, caller must re-read and decide } Callers that see false must re-read and decide whether to retry or abort their current sweep.`
2. **Monotonic SET (direction-aware).** If the expected direction of travel is always backward (this codebase's pattern), require the new value to be strictly LESS than the stored value:
`typescript async advanceBackward(contractAddress: string, blockNumber: number): Promise<boolean> { const result = await this.db.update(scannerState) .set({ lastScannedBlock: blockNumber, updatedAt: new Date() }) .where(and( eq(scannerState.contractAddress, contractAddress), sql`${scannerState.lastScannedBlock} > ${blockNumber}`,)) .returning({ id: scannerState.contractAddress }); return result.length > 0; } And a separate method for the "snap back to head" reset path. This encodes the scanner's direction-of-travel invariant at the DB level; a concurrent writer that tries to move backward from a position already past cannot succeed.`

Belt-and-braces: add a `scanner_lease(contractAddress, leaseHolder, expiresAt)` row, acquired via `INSERT ... ON CONFLICT DO UPDATE ... WHERE expires_at < NOW()` at the start of each scan loop, so at most one pod owns the scan at any moment. A rolling deploy then correctly drains; the new pod waits for the old pod's lease to expire.

Combine with a fix to the `updateFill` idempotency gap (idempotency key on `(orderHash, blockNumber, logIndex)`) so that even if a concurrent scanner does race through, `updateFill` cannot double-apply.



[L-05] Migration ledger abandoned with schema of record from `drizzle-kit` push output

Description

Multiple tables and columns declared in `packages/database/src/schema.ts` have no corresponding CREATE/ALTER migration in `packages/database/migrations/`. The ledger's `_journal.json` only references migrations 0000..0013, while `.sql` files exist through 0043, meaning `drizzle-kit migrate` would not apply the post-0013 files.

Neither `deploy.sh` nor `Dockerfile` invokes `drizzle-kit migrate` or `drizzle-kit push`. Schema synchronization is presumed to be invoked manually (e.g., `pnpm --filter @polygon-bot/database push`) out-of-band, and the migration ledger is not applied at deploy time. Production therefore stays consistent with `schema.ts` only when the operator remembers to run the out-of-band `push` step; the `migrations/` directory is not the authoritative source of the live schema.

The side effects of this design are worth noting even though production runtime is not affected:

Missing CREATE TABLE migrations:

- `fee_transfers` (declared at `schema.ts:386-419`): migrations 0023, 0037, 0038, and 0040 all ALTER this table but no migration creates it.
- `notification_history` (declared at `schema.ts:503-515`): migration 0040 alters `created_at` on a table that no migration creates.

Missing ADD COLUMN migrations:

- `users.is_banned` (schema.ts:29)
- `users.totp_iv` (schema.ts:39)
- `users.totp_auth_tag` (schema.ts:40)
- `copy_trading_follows.consecutive_failures` (schema.ts:250)
- `copy_trading_follows.last_failure_reason` (schema.ts:251)
- `copy_trading_follows.notifications_silenced` (schema.ts:252)
- `copy_trading_follows.auto_stopped_at` (schema.ts:256)
- `copy_trading_follows.auto_stopped_reason` (schema.ts:257)
- `copy_trading_follows.deleted_at` (schema.ts:258)

Grep of all 45 `.sql` files in `migrations/` returns zero matches for each.



Affected files

- `packages/database/migrations/_journal.json` : stops at entry 13 despite 45 `.sql` files existing.
- `packages/database/package.json:24` : exposes a `push` script (not `db:push`); no npm script invokes `drizzle-kit migrate` at deploy time.
- `Dockerfile` / `deploy.sh` : neither invokes `drizzle-kit migrate` nor `drizzle-kit push`.

Impact: Low

Production runtime is unaffected as long as the operator runs the out-of-band `push` step so the live DB matches `schema.ts`. The concrete costs are: (a) a `migrate`-based fresh-env bootstrap (DR drill, new region, dev onboarding from `migrations/`) fails, (b) schema history is not captured in version control in a form other tools can replay, (c) an operator who forgets the manual `push` step after a schema change produces drift that is invisible at deploy, and (d) a future engineer who reads the migrations directory and assumes it is authoritative will be misled.

Recommendations

Either commit to `drizzle-kit push` as the source of truth and remove the unused `migrations/` directory, or commit to the `migrate` workflow and backfill the missing DDL into new migration files (0044+). Mixing both is the worst of both worlds.

If keeping `push`, add a note at the top of `schema.ts` documenting that `migrations/` is historical and not applied at deploy time, and wire the `push` script into `deploy.sh` (or a CI step) so schema synchronization is no longer a manual, out-of-band action.

[L-06] `findByTokenSuffix` uses unvalidated input for leading wildcard LIKE on nonindexed column

Description

`findByTokenSuffix` builds a LIKE pattern by concatenating a user-controlled `suffix` after a literal `%`. The pattern is `'%' + suffix`, a leading wildcard. The B-tree indexes on `yesToken` / `noToken` (`schema.ts:86-87`) cannot be used for a leading-wildcard LIKE; absent a `pg_trgm` GIN index, the query degrades to a sequential scan. The function applies no length bound and does not escape the `%` / `_` LIKE metacharacters, so the caller-chosen string is interpreted as a pattern.

```
// packages/database/src/services/market-cache.ts:99-116
async findByTokenSuffix(suffix: string): Promise<MarketCache | null> {
  try {
    // Search for markets where yesToken or noToken ends with the suffix
    const result = await this.db.query.marketCached.findFirst({
```



```

        where: (marketCached, { or, sql }) =>
            or(
                sql`${marketCached.yesToken} LIKE ${'%' + suffix}`,
                sql`${marketCached.noToken} LIKE ${'%' + suffix}`
            ),
    });

    return result ? this.mapRowToMarketCache(result) : null;
} catch (error) {
    throw new DatabaseError(
        `Failed to find market by token suffix: ${error instanceof Error ?
error.message : 'Unknown error'}`
    );
}
}

```

Caller: `suffix` flows from the `/start_sell_{suffix}` deep-link payload with no prior validation:

```

// apps/bot-v2/src/index.ts:433-438
// Handle sell deep link: /start_sell_{last10CharsOfTokenId}
if (startPayload && startPayload.startsWith('sell_')) {
    const tokenSuffix = startPayload.replace('sell_', '');
    logger.info('Sell deep link received', { telegramId, tokenSuffix });
    const cachedMarket = await ctx.marketCacheService.findByTokenSuffix(tokenSuffix);
}

```

Two failure modes:

1. **Metacharacter injection.** A crafted link like `https://t.me/polygon_bot?start=sell_%` produces the pattern `LIKE '%'`, matching every row; `findFirst` returns an arbitrary market. `sell_____X` (nine `_` + X) matches any 10-character suffix ending in X. The user's sell form binds to the wrong market. A downstream position-ownership check in the caller (bot-v2:466) does blunt the economic impact (the user must hold a matching position), but the service is still a footgun for any future caller without that gate.
2. **Full-table scan DoS.** Short suffix (e.g., `sell_1`) triggers `LIKE '%1'`; the leading wildcard prevents the existing B-tree indexes on `yesToken` / `noToken` from being used, so the planner falls back to a sequential scan. Scan cost grows with `market_cached` table size; the DB pool is capped at 10 connections per `client/index.ts:7`. Repeated deep-link clicks from one attacker starve the pool for every other hot-path DB call.

The function's docstring says "last 10 characters," but nothing enforces that shape.

Affected files

- `packages/database/src/services/market-cache.ts:99-116` : unvalidated leading-wildcard LIKE pattern; B-tree indexes on `yesToken` / `noToken` (schema.ts:86-87) cannot serve this query shape.
- `apps/bot-v2/src/index.ts:433-438` : caller passing deep-link payload without pre-validation.



Impact: Low

Wrong-market binding is contained by the caller's position ownership check; the impact is UX confusion plus "Position not found" error messages. DoS amplification against the connection pool is the load bearing concern as the market cache grows.

Recommendations

Validate and escape the suffix at the service boundary:

```
async findByTokenSuffix(suffix: string): Promise<MarketCache | null> {
  // Enforce the documented shape: last 10 hex characters of a 32-byte token id
  if (!/^[0-9a-fA-F]{10}$/.test(suffix)) {
    throw new DatabaseError(`Invalid token suffix: ${suffix}`);
  }
  // Hex chars cannot include % or _, so LIKE metacharacter interpretation is moot after validation.
  // ...
}
```

Consider removing the function entirely and requiring deep-link producers to embed full token IDs so callers can use the indexed `findByTokenId`. If the short-suffix UX is required, add a trigram index on `yesToken` / `noToken` so the sequential scan cost is bounded.

[L-07] ResumeFollow does not clear auto-stop fields after manual resume in Telegram UI

Description

`resumeFollow` only flips `isActive = true`. It does not clear the auto-stop diagnostic fields. When a follow has been auto-stopped by `trackFailure` / `pauseSubscription` / `pauseAllUserSubscriptions`, those paths populate `autoStoppedAt`, `autoStoppedReason`, `consecutiveFailures`, `lastFailureReason`, and `notificationsSilenced`. Manually pressing "Resume" in the Telegram UI calls `resumeFollow`, which leaves all of those fields stale.

```
// packages/database/src/services/copy-trading.ts:287-301 (resumeFollow)
async resumeFollow(followId: number, followerTelegramId?: string): Promise<void> {
  try {
    const conditions = [eq(copyTradingFollows.id, followId),
      isNull(copyTradingFollows.deletedAt)];
    if (followerTelegramId) {
      conditions.push(eq(copyTradingFollows.followerTelegramId, followerTelegramId));
    }
    await this.db.update(copyTradingFollows)
      .set({ isActive: true, updatedAt: new Date() }) // ← only isActive
      .where(and(...conditions));
  }
```



```
    logger.info('Follow resumed', { followId });  
  } catch (error) { /* ... */ }  
}
```

Compare with `reactivateFollow`, which correctly resets all auto-stop states:

```
// packages/database/src/services/copy-trading.ts:556-574 (reactivateFollow reference)  
await this.db.update(copyTradingFollows)  
  .set({  
    isActive: true,  
    consecutiveFailures: 0,  
    lastFailureReason: null,  
    notificationsSilenced: false,  
    autoStoppedAt: null,  
    autoStoppedReason: null,  
    updatedAt: new Date(),  
  })  
  .where(and(eq(copyTradingFollows.id, followId), eq(copyTradingFollows.isActive, false)));
```

`reactivateFollow` has ZERO external callers; the entire monorepo references the name only at its definition site. The Telegram UI only exposes a single "Resume" button, which calls `resumeFollow`. So `resumeFollow` is the de facto single resume path for both manual pauses and auto-stops, yet it handles only the manual-pause case.

Two consequences follow:

1. View layer: `autoStoppedReason` is part of the `CopyTradingFollow` DTO returned by `mapFollowRow`. Any view that renders it displays a stale "10 consecutive failures: insufficient_balance" banner on a subscription that is actually active.
2. Next-failure re-trip: `consecutiveFailures` stays at 10 (or 20). `trackFailure` L491 checks `if (newConsecutiveFailures >= 10 && follow.isActive)` to trigger auto-stop. If the next copy-trade event fails with the same `lastFailureReason`, the counter hits 11 immediately and auto-stops again on the very first failure after resume.

Affected files

- `packages/database/src/services/copy-trading.ts:287-301` : `resumeFollow` with partial reset.
- `packages/database/src/services/copy-trading.ts:556-574` : `reactivateFollow` reference pattern (no external callers).

Impact: Low

User-facing state is incoherent: resumed subscriptions display stale auto-stop banners, and the consecutive-failure counter is not zeroed, so a single transient failure re-trips the auto-stop immediately. No fund loss. User experience only.



Recommendations

Merge the reset logic from `reactivateFollow` into `resumeFollow` so the single resume path handles both manual pause and auto-stop cases:

```
async resumeFollow(followId: number, followerTelegramId?: string): Promise<void> {
    const conditions = [eq(copyTradingFollows.id, followId),
    isNull(copyTradingFollows.deletedAt)];
    if (followerTelegramId) {
        conditions.push(eq(copyTradingFollows.followerTelegramId, followerTelegramId));
    }
    await this.db.update(copyTradingFollows)
        .set({
            isActive: true,
            consecutiveFailures: 0,
            lastFailureReason: null,
            notificationsSilenced: false,
            autoStoppedAt: null,
            autoStoppedReason: null,
            updatedAt: new Date(),
        })
        .where(and(...conditions));
    logger.info('Follow resumed and counters reset', { followId });
}
```

Optionally delete `reactivateFollow` (dead code) once `resumeFollow` subsumes it.

[L-08] Max subscriptions per user not enforced due to race condition in follow method

Description

The `follow()` method enforces `MAX_SUBSCRIPTIONS_PER_USER` via a read-then-write pattern with no transaction and no row-level lock on the user record. Two concurrent `follow()` calls for the same user (to different leaders) can both observe `count = 9`, both pass the guard, and both insert, producing 11 or more subscriptions. No DB-level constraint enforces the cap.

```
// packages/database/src/services/copy-trading.ts:182-206
// Check max subscriptions limit
const currentFollowCount = await this.getFollowingCount(followerTelegramId);
if (currentFollowCount >= CopyTradingService.MAX_SUBSCRIPTIONS_PER_USER) {
    throw new DatabaseError(`Maximum ${CopyTradingService.MAX_SUBSCRIPTIONS_PER_USER}
subscriptions allowed per user`);
}

// Generate unique share code
const shareCode = generateShareCode();

const [follow] = await this.db.insert(copyTradingFollows).values({
    followerTelegramId,
    leaderAddress: leaderAddress.toLowerCase(),
});
```



```
leaderTelegramId: leaderTelegramId ?? null,  
leaderName: leaderName ?? null,  
mode,  
multiplier: multiplier?.toString() ?? '1',  
// ...  
shareCode,  
}).returning();
```

The call sequence is `SELECT COUNT → compare → INSERT`. The two statements execute at read-committed isolation against uncommitted concurrent inserts, and no `SELECT ... FOR UPDATE` on the user row serializes them. `follow()` is reachable from the Telegram UI "Confirm" button (rapid taps), from the share-code deeplink handler in `apps/bot-v2/src/index.ts:929`, and from any admin or future callers. The service layer cannot rely on UI throttling.

Affected files

- `packages/database/src/services/copy-trading.ts:182-206` : non-atomic count and insert.
- `packages/database/src/schema.ts` : no CHECK constraint, trigger, or partial unique index enforcing the cap.

Impact: Low

Product policy cap is a soft guardrail rather than a security boundary. N concurrent `follow()` calls from the same user yield up to `cap - 1 + N` subscriptions. Resource impact on the listener/preprocessor is small (10→14 is not a DoS vector); the primary effect is that users can circumvent the stated limit by racing the create flow.

Recommendations

Either (a) wrap the count and insert in a transaction that takes `SELECT ... FOR UPDATE` on the `users` row (serializing concurrent follows for the same user), or (b) enforce the cap at the DB via a trigger that counts active follows on INSERT. Option (a) is simpler:

```
return await this.db.transaction(async (tx) => {  
  // Lock the user row to serialize concurrent follow() attempts  
  await tx.execute(sql`SELECT 1 FROM users WHERE telegram_id = ${followerTelegramId} FOR UPDATE`);  
  
  const currentFollowCount = await tx.select({ count: sql<number>`COUNT(*)` })  
    .from(copyTradingFollows)  
    .where(and(  
      eq(copyTradingFollows.followerTelegramId, followerTelegramId),  
      isNull(copyTradingFollows.deletedAt)  
    ));  
  if (Number(currentFollowCount[0]?.count ?? 0) >=  
    CopyTradingService.MAX_SUBSCRIPTIONS_PER_USER) {  
    throw new DatabaseError(`Maximum ${CopyTradingService.MAX_SUBSCRIPTIONS_PER_USER}  
      subscriptions allowed per user`);  
  }  
}
```




```
const [follow] = await tx.insert(copyTradingFollows).values({ /* ... */}).returning();
return follow;
});
```

[L-09] `recordCopyTrade` lacks idempotency key causing duplicate event in aggregate

Description

`recordCopyTrade` inserts into `copy_trade_history_2` with no uniqueness guard involving `leaderTxHash`. The schema indexes on `followerTelegramId`, `followId`, and composite `(followId, ...)` tuples for analytics, but none of these prevent the same leader event from being recorded twice.

```
// packages/database/src/services/copy-trading.ts:1244-1298
async recordCopyTrade(params: RecordCopyTradeParams): Promise<CopyTradeHistoryRecord> {
  try {
    const [record] = await this.db.insert(copyTradeHistory2).values({
      followId: params.followId,
      leaderAddress: params.leaderAddress.toLowerCase(),
      followerTelegramId: params.followerTelegramId,
      marketId: params.marketId,
      // ...
      tradeType: params.tradeType,
      // ...
      leaderTxHash: params.leaderTxHash ?? null,
      source: params.source ?? null,
    }).returning();
    // ...
  }
}
```

```
// packages/database/src/schema.ts:322-366 (copyTradeHistory2)
export const copyTradeHistory2 = pgTable('copy_trade_history_2', {
  // ...
  leaderTxHash: text('leader_tx_hash'),
  source: text('source'), // 'pending' | 'mined'
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow().notNull(),
}, (table) => ({
  followerIdx: index('idx_copy_history2_follower').on(table.followerTelegramId),
  followIdIdx: index('idx_copy_history2_follow_id').on(table.followId),
  followDailyIdx: index('idx_copy_history2_follow_daily').on(table.followId,
    table.tradeType, table.status, table.createdAt),
  followMarketIdx: index('idx_copy_history2_follow_market').on(table.followId,
    table.marketId, table.tradeType, table.status),
  // no unique constraint on leader_tx_hash
}));
```

Upstream, `apps/copy-trade-listener/src/process-event.ts` uses a Redis `SET NX EX` keyed on `(txHash, logIndex)` to deduplicate within a TTL window. Once the TTL expires, the block scanner (or a reorg replay, or a manual backfill script that bypasses NATS) can re-



enqueue the same event: the Redis key is gone, NATS's `max_deliver: 1` applies only per message, and the BullMQ scheduler path has no equivalent deduplication. The second delivery reaches `recordCopyTrade` and creates a duplicate row.

Duplicates inflate every aggregate over `copy_trade_history_2`:

- `getDailySpent` (L773-797, `SUM(followerAmount)`): user can be prematurely locked out of the deprecated DB-based daily limit path.
- `getMarketSpent` (L804-829), `getMarketSpentTotal` (L835-856), `getDailySpentOnMarket` (L1352-1378).
- `getFollowStats` (L1437-1488, `totalVolume` and `successfulTrades`): surfaces directly in the Telegram UI.
- `getFollowProfit` (L1494-1546): surfaces in user-visible statistics.

The in-file comments acknowledge that the DB-aggregate path is deprecated in favor of Redis counters, but `checkDailyLimit` (L861-886) still calls `getDailySpent`, and the UI handlers have 30+ call sites for the stats aggregates.

Affected files

- `packages/database/src/services/copy-trading.ts:1244-1298`: `recordCopyTrade` inserts without idempotency.
- `packages/database/src/schema.ts:322-366`: `copy_trade_history_2` is missing a unique constraint on `(leader_tx_hash, follower_telegram_id, token_id, trade_type)`.

Impact: Low

Data integrity on user-visible statistics and on the deprecated DB-based daily-limit path. Duplicates inflate volume/profit numbers shown in Telegram and can prematurely trip `checkDailyLimit`. The modern Redis-counter path is the primary spend gate, so real-money double spending is not directly caused by this.

Recommendations

Add a partial unique index that tolerates null `leaderTxHash` (e.g., on-chain source only) while enforcing idempotency when it is present:

```
CREATE UNIQUE INDEX idx_copy_history2_dedup
ON copy_trade_history_2 (leader_tx_hash, follower_telegram_id, token_id, trade_type)
WHERE leader_tx_hash IS NOT NULL
AND status IN ('success', 'pending');
```

In `recordCopyTrade`, use `.onConflictDoNothing({ target: [...] })` and return `null` to the caller when the row is a duplicate, matching the pattern already used for `fee_transfers.create`.



[L-10] `getFollowById` and `toggleNotificationsMuted` do not filter soft-deleted follows

Description

Both overloads of `getFollowById` omit the `isNull(deletedAt)` predicate. This function is the ownership gate used by the majority of Telegram UI callbacks in `packages/telegram-ui-v2/src/handlers/copy-trading.ts`. `toggleNotificationsMuted` also omits the filter in its own WHERE clause; even if a caller checks ownership via `getFollowById`, the mutator happily writes to a soft-deleted row.

```
// packages/database/src/services/copy-trading.ts:1052-1075 (getFollowById overloads)
async getFollowById(followId: number): Promise<CopyTradingFollow | null>;
async getFollowById(followerTelegramId: string, followId: number): Promise<CopyTradingFollow | null>;
async getFollowById(followIdOrTelegramId: number | string, followId?: number):
Promise<CopyTradingFollow | null> {
  try {
    if (typeof followIdOrTelegramId === 'number') {
      // Called with just followId
      const follow = await this.db.query.copyTradingFollows.findFirst({
        where: (table, { eq }) => eq(table.id, followIdOrTelegramId), // ← no
deletedAt filter
      });
      return follow ? this.mapFollowRow(follow) : null;
    } else {
      // Called with telegramId and followId
      const follow = await this.db.query.copyTradingFollows.findFirst({
        where: (table, { eq, and }) => and(
          eq(table.followerTelegramId, followIdOrTelegramId),
          eq(table.id, followId!) // ← no
deletedAt filter
        ),
      });
      return follow ? this.mapFollowRow(follow) : null;
    }
  } catch (error) { /* ... */ }
}
```

```
// packages/database/src/services/copy-trading.ts:430-446 (toggleNotificationsMuted)
async toggleNotificationsMuted(followId: number, muted: boolean, followerTelegramId?:
string): Promise<boolean> {
  try {
    const conditions = [eq(copyTradingFollows.id, followId)];
    if (followerTelegramId) {
      conditions.push(eq(copyTradingFollows.followerTelegramId, followerTelegramId));
    }
    await this.db.update(copyTradingFollows)
      .set({ notificationsMuted: muted, updatedAt: new Date() })
      .where(and(...conditions)); // ← no
deletedAt filter
  }
}
```



```
// ...  
}  
}
```

Concretely, a user who has unfollowed a leader but retains an old Telegram inline-keyboard callback (pinned/forwarded message, scroll history) can click the mute toggle on the already-deleted follow. The ownership check passes (user owns the row regardless of `deletedAt`); `toggleNotificationsMuted` happily writes. The row stays soft-deleted and never fires a copy-trade, so the state change is inert; but the UI layer then renders a "subscription details" view for the deleted follow (handlers route through `getFollowById` → `getSubscriptionDetailsView`), producing a ghost subscription that the user thought they had already removed.

`mapFollowRow` does not return `deletedAt`, so the handler has no way to detect the deleted state from the returned DTO.

Affected files

- `packages/database/src/services/copy-trading.ts:1052-1075` : both `getFollowById` overloads omit `isNull(deletedAt)` .
- `packages/database/src/services/copy-trading.ts:430-446` : `toggleNotificationsMuted` omits `isNull(deletedAt)` .

Impact: Low

Own-data only; no cross-user disclosure, no fund risk. Users can resurrect mute state on subscriptions they already deleted, and stale callbacks render UI for ghost rows, producing a confusing user experience. The ghost rows never trigger copy-trades, so the economic impact is nil.

Recommendations

Add `isNull(copyTradingFollows.deletedAt)` to both overloads of `getFollowById` and to the WHERE clause of `toggleNotificationsMuted` . This is the same pattern `getFollowByLeaderAddress` (L1082-1088) already applies correctly. Optionally surface `deletedAt` through `mapFollowRow` so handlers can short-circuit with a "this follow has been removed" message instead of silently returning null.

[L-11] Referral_commissions has no UNIQUE on (trade_id beneficiary_id level)

Description

The `referralCommissions` table declares three indexes; none are unique:



```
// schema.ts:90-107
export const referralCommissions = pgTable('referral_commissions', {
  id: uuid('id').defaultRandom().primaryKey(),
  tradeId: text('trade_id').notNull(),
  traderId: uuid('trader_id').notNull(),
  beneficiaryId: uuid('beneficiary_id').notNull(),
  level: integer('level').notNull(),
  commissionPercent: decimal('commission_percent', { precision: 5, scale: 2 }).notNull(),
  commissionAmount: decimal('commission_amount', { precision: 18, scale: 6 }).notNull(),
  tradeFeeAmount: decimal('trade_fee_amount', { precision: 18, scale: 6 }).notNull(),
  status: text('status').default('pending').notNull(),
  txHash: text('tx_hash'),
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow().notNull(),
  transferredAt: timestamp('transferred_at', { withTimezone: true }),
}, (table) => ({
  beneficiaryIdx: index('idx_commissions_beneficiary').on(table.beneficiaryId),
  traderIdx: index('idx_commissions_trader').on(table.traderId),
  statusIdx: index('idx_commissions_status').on(table.status),
}));
```

No `uniqueIndex` or composite UNIQUE on the natural identity tuple `(tradeId, beneficiaryId, level)`. The service's `create()` method uses a plain INSERT with no `.onConflictDoNothing()`. On BullMQ redelivery of a partially completed commission-recording job, a second INSERT with the same `(tradeId, beneficiaryId, level)` succeeds and produces a duplicate row.

The missing UNIQUE is the storage-shape half of the commission dual-ledger integrity gap. The API-shape half is also missing: the commission INSERT and the `addPendingCommission` call are not wrapped in a transaction. Either alone is incomplete; both must be fixed.

Affected files

- `packages/database/src/schema.ts:90-107`: `referralCommissions` table, missing UNIQUE on `(trade_id, beneficiary_id, level)`.
- `packages/database/src/services/commissions.ts:40-76`: `create` plain INSERT, no `.onConflictDoNothing()`.

Impact: Low

Two distinct INSERTs with the same `(tradeId, beneficiaryId, level)` succeed and produce duplicate rows. The duplicates pay the referrer twice for one trade. No DB-layer guard rejects them; the only protection is application-layer caller discipline that today is absent. Drift accumulates silently across worker crashes and BullMQ redeliveries.

Recommendations

Add a unique index in a migration:

```
// schema.ts:103-107
}, (table) => ({
  beneficiaryIdx: index('idx_commissions_beneficiary').on(table.beneficiaryId),
```



```
traderIdx: index('idx_commissions_trader').on(table.traderId),
statusIdx: index('idx_commissions_status').on(table.status),
uniqueCommission: uniqueIndex('idx_commissions_unique')
.on(table.tradeId, table.beneficiaryId, table.level),
));
```

Then update `commissionService.create()` to use `.onConflictDoNothing().returning()` and return `null` on duplicates, mirroring the pattern in `feeTransferService.create()` at `fee-transfers.ts:65-85`. Callers can short-circuit on `null`.

Pre-migration: audit existing data for already duplicated rows:

```
SELECT trade_id, beneficiary_id, level, count(*) AS dup_count
FROM referral_commissions
GROUP BY trade_id, beneficiary_id, level
HAVING count(*) > 1;
```

If duplicates exist, deduplicate (keep the earliest by `created_at`, sum the others into a reconciliation report, optionally credit/debit `users.pending_commission` to compensate) before adding the constraint; otherwise, the migration fails.

[L-12] Self-follow guard only fires when both parties are in-protocol users

Description

The self-follow check in `follow()` is a truthy gate on `leaderTelegramId`:

```
// copy-trading.ts:171-174
// Prevent self-follow (would cause infinite copy-trade loop)
if (leaderTelegramId && followerTelegramId === leaderTelegramId) {
  throw new DatabaseError('Cannot follow your own wallet');
}
```

The `===` only evaluates when `leaderTelegramId` is truthy. When the caller does not supply the leader's telegram ID (the standard external-leader case where the leader is just a Polygon address with no associated bot user), the check is skipped entirely. The follow row is created with the user's own Safe address as the leader.

Once that row exists, the copy-trade listener pipeline treats every trade signed by that Safe as a "leader trade" and triggers a follower reaction in the same Safe. Each reaction is itself a trade signed by the same Safe, which the listener picks up as a fresh leader event. The cycle continues until `dailyLimit`, `singleTradeLimit`, or the user's USDC balance breaks it.

The schema also does not prevent this:



```
// schema.ts:209-213
followerTelegramId: text('follower_telegram_id').references(() =>
users.telegramId).notNull(),
leaderAddress: text('leader_address').notNull(), // Gnosis Safe or EOA address
leaderTelegramId: text('leader_telegram_id'), // NULL if external leader
```

No constraint that `leaderAddress != users.gnosisSafeAddress` for the follower's row.

Affected files

- `packages/database/src/services/copy-trading.ts:171-174` : truthy gate self-follow check.
- `packages/database/src/services/copy-trading.ts:150-228` : `follow()` write path.
- `packages/database/src/schema.ts:209-213` : `copyTradingFollows` columns, no cross-field constraint.

Impact: Low

A user who follows their own Safe via the external-leader path (deeplink share code, paste-in-address dialog, any flow where `leaderTelegramId` is undefined) creates a copy-trade feedback loop. Each reflected trade burns slippage + bot fee + relayer cost. Bounded by the cap fields and the user's balance (the loop eventually halts), but the balance bleeds out one fee at a time until it does. This requires the handler to route the follow through the external-leader path; current handlers for list-selection paths do pass the leader's telegram ID and the existing check fires.

Recommendations

Add a second check that compares `leaderAddress` against the follower's own `gnosis_safe_address`. This fires regardless of whether the leader is in-protocol or external:

```
// copy-trading.ts:171-174
const follower = await this.db.query.users.findFirst({
  where: (u, { eq }) => eq(u.telegramId, followerTelegramId),
  columns: { gnosisSafeAddress: true },
});
const followerSafe = follower?.gnosisSafeAddress?.toLowerCase();
const leaderAddrNorm = leaderAddress.toLowerCase();

if (leaderTelegramId && followerTelegramId === leaderTelegramId) {
  throw new DatabaseError('Cannot follow your own wallet');
}
if (followerSafe && leaderAddrNorm === followerSafe) {
  throw new DatabaseError('Cannot follow your own wallet');
}
```

The address-based check is unconditional. The existing telegramId-based check is preserved for the case where the user follows another bot user whose Safe address differs from their own. Both `.toLowerCase()` normalizations match the existing pattern used at line 193 in the INSERT.



Belt-and-braces at the DB layer: a trigger that rejects inserts where `leader_address` equals the follower's `gnosis_safe_address`. It is harder to express cleanly in Drizzle because it references another table; it is probably a raw SQL trigger in a migration.

[L-13] Copy trading follows unique index lack where `deleted_at` is null

Description

The unique index on `copy_trading_follows` has no `WHERE` predicate, so it includes soft-deleted rows:

```
// schema.ts:262-270
}, (table) => ({
  followerIdx: index('idx_copy_follows_follower').on(table.followerTelegramId),
  leaderAddrIdx: index('idx_copy_follows_leader_addr').on(table.leaderAddress),
  activeIdx: index('idx_copy_follows_active').on(table.isActive),
  leaderActiveIdx: index('idx_copy_follows_leader_active').on(table.leaderAddress,
table.isActive, table.deletedAt),
  // Ensure follower can only follow same leader once
  uniqueFollow: uniqueIndex('idx_copy_follows_unique').on(table.followerTelegramId,
table.leaderAddress),
}));
```

Drizzle generates a plain `CREATE UNIQUE INDEX ... (follower_telegram_id, leader_address)` that applies to all rows regardless of soft-delete status.

Soft delete is done by `unfollow()`:

```
// copy-trading.ts:243-263
async unfollow(followerTelegramId: string, leaderAddress: string): Promise<void> {
  try {
    await this.db.update(copyTradingFollows)
      .set({
        isActive: false,
        deletedAt: new Date(),
        updatedAt: new Date(),
      })
      .where(and(
        eq(copyTradingFollows.followerTelegramId, followerTelegramId),
        eq(copyTradingFollows.leaderAddress, leaderAddress.toLowerCase())
      ));
  }
}
```

`follow()` then attempts a fresh INSERT. The `(followerTelegramId, leaderAddress)` pair collides with the soft-deleted row → PG raises `23505` → the catch at lines 222-225 converts it to "Already following this trader":

```
// copy-trading.ts:220-227
} catch (error: any) {
  if (error instanceof DatabaseError) throw error;
```




```
if (error?.code === '23505' && error?.constraint?.includes('copy_follows_unique')) {  
  throw new DatabaseError('Already following this trader');  
}  
throw new DatabaseError(`Failed to follow trader: ${error instanceof Error ?  
error.message : 'Unknown error'}`);  
}
```

Affected files

- `packages/database/src/schema.ts:269` : `uniqueFollow` unique index without `WHERE deleted_at IS NULL`.
- `packages/database/src/services/copy-trading.ts:243-263` : `unfollow` soft-deletes via `deletedAt`.
- `packages/database/src/services/copy-trading.ts:191-226` : `follow` INSERT, catch mapping `23505` to misleading error.

Impact: Low

Once a user soft-unfollows a leader, they cannot re-follow that same leader. They receive the misleading "Already following this trader" message even though they just unfollowed. This is a pure UX defect, with no security implication and no data corruption. However, it is reproducible on every unfollow-then-refollow sequence for the same `(follower, leader)` pair, which is a common user flow.

Recommendations

Preferred: partial unique index. One-line schema change plus a migration:

```
// schema.ts:269  
uniqueFollow: uniqueIndex('idx_copy_follows_unique')  
  .on(table.followerTelegramId, table.leaderAddress)  
  .where(sql`deleted_at IS NULL`),
```

Generated SQL:

```
CREATE UNIQUE INDEX idx_copy_follows_unique  
ON copy_trading_follows (follower_telegram_id, leader_address)  
WHERE deleted_at IS NULL;
```

After the migration, soft-deleted rows are excluded from the uniqueness check; re-follow inserts a fresh row with no collisions.

Alternative: UPDATE-reactivate inside `follow()`. Detect a soft-deleted row first and UPDATE it back to active (clear `deletedAt`, set new mode / amount / etc., bump `updatedAt`) instead of inserting a new row. Preserves the original `followId` and history references. Slightly nicer semantically but costs more code than the partial index fix.



[L-14] Record copy trade error handler spreads drizzle internals to stdout

Description

The catch block inside `recordCopyTrade` emits a `console.error` whose payload is the spread of the entire error object:

```
// copy-trading.ts:1283-1297
} catch (error) {
  if (error instanceof DatabaseError) throw error;

  // Log full error details for debugging
  console.error('[recordCopyTrade] Database error:', error);
  console.error('[recordCopyTrade] Error details:', {
    name: error instanceof Error ? error.name : 'unknown',
    message: error instanceof Error ? error.message : String(error),
    stack: error instanceof Error ? error.stack : undefined,
    // Log drizzle-specific error details if available
    ...(error && typeof error === 'object' ? error : {}) // ← spreads full error
  });

  throw new DatabaseError(`Failed to record copy trade: ${error instanceof Error ?
error.message : 'Unknown error'}`);
}
```

Two issues compound:

1. `...error` spread. For a Drizzle wrapping of `postgres-js` errors, the enumerable keys on the error object include `code`, `severity`, `detail`, `hint`, `position`, `internalQuery`, `where`, `schema`, `table`, `column`, `dataType`, `constraint`, `file`, `line`, `routine`, `query`, and `parameters`. The `query` field is the raw SQL with `$1` / `$2` placeholders; `parameters` is the array of values that would be bound. For `recordCopyTrade`, those values include `followId`, `leaderAddress` (lowercased Ethereum address), `followerTelegramId` (Telegram user ID), `marketId`, `tokenId`, `tradeAmount` (USDC), `leaderTxHash`, etc.

2. `console.error` instead of `logger.error`. This file imports `logger` from `@polygun-bot/shared` and uses it in most other places. The two `console.error` calls at lines 1287 and 1288 bypass any Pino transport, redact paths, or log-level filtering configured for the structured logger. They go straight to stdout / stderr, which Docker captures into the container log driver, which forwards to whatever aggregation backend exists (Datadog, Loki, CloudWatch, S3).

The error path fires on any DB error during copy-trade recording: connection drops, FK violations, lock timeouts, statement timeout (the 30 s cap configured in `client/index.ts:14`).



Affected files

- `packages/database/src/services/copy-trading.ts:1283-1297` : `recordCopyTrade` catch block uses `console.error` + `...error` spread.

Impact: Low

Every DB error in the copy-trade recording path leaks the raw SQL plus the bound parameters (user telegram IDs, wallet addresses, token IDs, amounts, leader tx hashes) to stdout. The log lines accumulate in whatever retention the log aggregation backend has. Anyone with log-read access (operators, on-call, support, the log vendor) can reconstruct which users traded, which markets, which leaders they follow, and what amounts: all data that should have been redacted. Not attacker-triggerable; triggered by any transient DB instability or schema migration.

Recommendations

Replace the spread with a focused projection through the structured logger:

```
// copy-trading.ts:1283-1297
} catch (error) {
  if (error instanceof DatabaseError) throw error;

  const dbErr = error as { code?: string; constraint?: string; column?: string; table?: string };
  logger.error({
    message: error instanceof Error ? error.message : String(error),
    code: dbErr.code,
    constraint: dbErr.constraint,
    column: dbErr.column,
    table: dbErr.table,
    // Do NOT include `query`, `parameters`, `detail`, or the full error.
  }, '[recordCopyTrade] failed');

  throw new DatabaseError(`Failed to record copy trade: ${error instanceof Error ? error.message : 'Unknown error'}`);
}
```

Hardening pass across the file:

- `mapFollowRow` at line 1682 uses a similar `console.log` for percentage-mode debug; same logger-bypass concern. Replace with `logger.debug`.
- Repo-wide grep for `\\.\\.\\.\\.error` and `\\.\\.\\.\\.error` to find any other spread patterns.

Belt-and-braces: configure Pino's `redact.paths` with wildcards like `*.query`, `*.parameters`, `*.detail` so any future spread caught by the structured logger is sanitized at the transport layer. This does not protect the `console.*` paths, which is why those must go.



[L-15] Asymmetric soft-delete in `getReferralChain` disrupts F3 commission payouts

Description

`getReferralChain` in `packages/database/src/services/referrals.ts` walks upward from a trader to find referrers who should receive commissions. When fetching each referrer, it filters out soft-deleted users:

```
// referrals.ts:60-67
const referrer = await this.db
  .select()
  .from(users)
  .where(and(
    eq(users.id, current[0].referredById),
    isNull(users.deletedAt) //@audit-info skips soft-deleted referrers
  ))
  .limit(1);

if (!referrer[0]) break; //@audit-info stops the entire chain walk
```

The problem is the `break`; if the F2 referrer is soft-deleted, the chain walk stops entirely. A live F3 referrer above the deleted F2 is never reached and silently loses their commission. Example: Trader -> F1 (active) -> F2 (soft-deleted) -> F3 (active). F1 receives their commission, but F2 being deleted causes `break`, so F3 is never visited.

Meanwhile, `getNetworkStats` uses a recursive CTE that walks **downward** and also filters `deleted_at IS NULL`:

```
// referrals.ts:88-98
WITH RECURSIVE referral_tree AS (
  SELECT id, referred_by_id, 1 as level
  FROM users
  WHERE referred_by_id = ${userId} AND deleted_at IS NULL
  UNION ALL
  SELECT u.id, u.referred_by_id, rt.level + 1
  FROM users u
  INNER JOIN referral_tree rt ON u.referred_by_id = rt.id
  WHERE u.deleted_at IS NULL AND rt.level < 3
)
```

Here, if the deleted user is an intermediate node, the CTE still continues through other branches and **does count F3** in the network size. So the user sees F3 in their network stats, but F3 never receives commissions, causing an inconsistency. It is best to add consistency across design choices and apply the same choice to continue the walk or terminate depending on your design decision.



[L-16] Deposit checker overlapping runs can send duplicate reminders

Description

The deposit checker runs `checkAndSendReminders()` on a `setInterval` polling loop without any overlap guard (running flag or distributed lock). If the interval fires while a previous run is still processing (e.g., due to slow Telegram API responses or database latency), two concurrent runs can observe the same set of users and attempt to send reminder notifications before either run marks them as notified in Redis via `wasAlreadyNotified()`.

The deposit checker identifies users who have registered a Safe address but have not yet made a trade (via a `LEFT JOIN` against `fee_transfers` where `ft.id IS NULL`), then sends Telegram reminders. Redis `SET NX`-based deduplication is used to prevent re-sending, but the check-then-send-then-mark sequence is not atomic, creating a race condition.

Location:

- `apps/deposit-checker/src/index.ts` — `checkAndSendReminders()`

Reachability / endpoint: Normal operation under load or slow Telegram API responses.

Vulnerable flow:

```
Run 1: query users needing 5-min reminder → [U1, U2, U3]
      → wasAlreadyNotified(U1) = false → send Telegram message...
Run 2 (interval fires): query users → [U1, U2, U3] // same result
      → wasAlreadyNotified(U1) = false (Run 1 hasn't marked yet)
      → send Telegram message to U1 → duplicate notification
Run 1: mark U1 as notified in Redis
Run 2: mark U1 as notified (no-op, already set)
```

Affected code:

```
// apps/deposit-checker/src/index.ts

const CHECK_INTERVAL_MS = 60 * 1000; // 1 minute
const DELAY_BETWEEN_MESSAGES = 5000; // 5 seconds between messages

async function checkAndSendReminders(
  db: DbClient,
  redis: Redis,
  botToken: string
): Promise<void> {
  // No running flag, no lock acquisition

  // — Process 5-minute reminders (Tutorial) —
  const users5min = await getUsersNeeding5minReminder(db);

  for (const user of users5min) {
    if (await wasAlreadyNotified(redis, REDIS_PREFIX_5MIN, user.telegramId)) {
      continue;
    }
  }
}
```



```
    }
    const result = await sendPhotoMessage(botToken, user.telegramId, TUTORIAL_THUMBNAIL_URL,
message5min);
    if (result.success) {
        await markAsNotified(redis, REDIS_PREFIX_5MIN, user.telegramId, REDIS_TTL_5MIN);
        await recordNotification(db, user.telegramId, 'deposit_reminder_5min', 'sent');
    }
    await new Promise(resolve => setTimeout(resolve, DELAY_BETWEEN_MESSAGES)); // 5s delay
}

// — Process 15-minute reminders (Wallets to copy) —
const users15min = await getUsersNeeding15minReminder(db);
// ... same pattern ...
}

// getUsersNeeding5minReminder uses LEFT JOIN on fee_transfers to find users with no trades:
async function getUsersNeeding5minReminder(db: DbClient): Promise<UserNeedingReminder[]> {
    const result = await db.execute(sql`
        SELECT u.telegram_id, u.created_at
        FROM users u
        LEFT JOIN fee_transfers ft ON u.telegram_id = ft.telegram_id
        WHERE u.deleted_at IS NULL
            AND u.gnosis_safe_address IS NOT NULL
            AND u.gnosis_safe_address != ''
            AND u.created_at <= NOW() - INTERVAL '5 minutes'
            AND u.created_at > NOW() - INTERVAL '15 minutes'
            AND u.created_at >= ${CUTOFF_DATE}::timestamp
            AND ft.id IS NULL
        GROUP BY u.telegram_id, u.created_at
    `);
    // ...
}
```

Redis deduplication helpers:

```
async function wasAlreadyNotified(redis: Redis, prefix: string, telegramId: string):
Promise<boolean> {
    const key = `${prefix}${telegramId}`;
    const exists = await redis.exists(key);
    return exists === 1;
}

async function markAsNotified(redis: Redis, prefix: string, telegramId: string, ttlSeconds:
number): Promise<void> {
    const key = `${prefix}${telegramId}`;
    await redis.setex(key, ttlSeconds, '1');
}
```

Impact:

Users receive duplicate Telegram deposit reminder messages. This is a UX annoyance, not a financial vulnerability. Under sustained load, the probability of race conditions increases.



Recommendations

Add a running flag to prevent overlapping interval fires, and make the Redis deduplication atomic with the notification send:

```
let running = false;
setInterval(async () => {
  if (running) return;
  running = true;
  try { await checkAndSendReminders(db, redis, botToken); }
  finally { running = false; }
}, CHECK_INTERVAL_MS);
```

Alternatively, combine the `wasAlreadyNotified` check and the `markAsNotified` into an atomic `SET NX EX` call BEFORE sending the message. If the SET succeeds, send; if not, skip. This eliminates the race window entirely:

```
const marked = await redis.set(notifyKey(user.telegramId), '1', 'EX', ttlSeconds, 'NX');
if (marked !== 'OK') continue; // already being handled
await sendReminderMessage(botToken, user.telegramId, ...);
```

[L-17] ClaimPromo string comparison can fail after transfer leading to replay loss risk

Description

`KolPromoService.claimPromo` at `packages/database/src/services/kol-promo.ts:194` performs strict string equality:

```
if (amount !== kolPromo.claimableAmount) throw ...
```

`kolPromo.claimableAmount` is `numeric(18,6)` (`schema.ts:468`) and is returned in fixed-scale text form (`"20.000000"`), while callers commonly produce normalized strings like `"20"`.

The check runs after successful on-chain transfer and receipt verification in the worker. If the string comparison fails, the worker reverts status from `processing` to `pending`. This creates a payout/accounting mismatch and potential replay window. However, replay is constrained by deterministic `jobId` reuse and queue retention behavior (`packages/telegram-ui-v2/src/handlers/promo.ts`, `packages/queue/src/queues/trading.queue.ts`), so this is not an unconstrained per-click infinite drain.

Recommendations

Normalize comparison with fixed-point semantics:



```
import { BigNumber } from 'bignumber.js';

if (!new BigNumber(amount).eq(kolPromo.claimableAmount)) {
  throw new DatabaseError('Claim amount mismatch');
}
```

Also enforce replay resistance at the state layer (single-claim idempotency keyed by `userPromoId` + settled transfer marker).

[L-18] Fee_transfers.trade_type is unconstrained text type safety is TS only

Description

The `trade_type` column on `fee_transfers` is declared as plain text with the union documented only in a comment:

```
// schema.ts:386-394 (relevant slice of feeTransfers)
export const feeTransfers = pgTable('fee_transfers', {
  id: serial('id').primaryKey(),
  telegramId: text('telegram_id').notNull(),

  // Trade context
  tradeType: text('trade_type').notNull(), // 'buy' | 'sell' | 'claim' | 'auto_claim' |
  'copy_buy' | 'copy_sell' | 'limit_fill'
  tradeId: text('trade_id'),
  // ...
});
```

The TS union lives in the service file:

```
// fee-transfers.ts:8
export type FeeTradeType = 'buy' | 'sell' | 'claim' | 'auto_claim' | 'copy_buy' |
'copy_sell' | 'limit_fill';
```

`FeeTransferService.create` accepts `tradeType: FeeTradeType` and is type-safe at compile time. However:

1. Any caller can `as FeeTradeType` cast a string of their choice and bypass the union.
2. Any caller can call `db.insert(feeTransfers).values({ tradeType: 'arbitrary_string', ... })` directly without going through the service.

Both bypass routes write the row and PG accepts it. Downstream reporting that filters by the documented union members silently excludes the corrupt rows. This is already happening in practice: the limit-order NATS consumer writes `tradeType: isBuy ? 'limit_buy' : 'limit_sell'` instead of `'limit_fill'`, and the rows land in `fee_transfers` without complaint.



Affected files

- `packages/database/src/schema.ts:386-394` : `feeTransfers.tradeType` unconstrained text, intended union documented only as a comment.
- `packages/database/src/services/fee-transfers.ts:8` : `FeeTradeType` TS union (enforced in memory only).

Impact: Low to Medium (Low per row, Medium aggregate across millions of fee rows)

Analytics, reconciliation, and per-trade-type reporting filter by the documented union members and silently exclude rows with values outside it. A `SELECT trade_type, SUM(fee_amount) FROM fee_transfers WHERE trade_type IN (...)` undercounts revenue proportionally to how many corrupt rows exist; the limit-order NATS consumer writing `limit_buy` / `limit_sell` (versus the documented `limit_fill`) is a live example already producing corrupt rows in production. `SELECT DISTINCT trade_type FROM fee_transfers` surfaces the unexpected values but only if someone runs it. There is no fund impact and no integrity risk for user operations, but this is an active data corruption defect that scales across the fee ledger and compounds silently until someone notices the revenue gap, which is why the overall severity is Medium rather than Low.

Recommendations

Preferred: convert to a Drizzle `pgEnum` :

```
// schema.ts (top of file)
import { pgEnum } from 'drizzle-orm/pg-core';

export const feeTradeTypeEnum = pgEnum('fee_trade_type', [
  'buy', 'sell', 'claim', 'auto_claim', 'copy_buy', 'copy_sell', 'limit_fill',
]);

// Inside feeTransfers:
tradeType: feeTradeTypeEnum('trade_type').notNull(),
```

Generates a real Postgres enum type. Bad inserts throw `invalid input value for enum fee_trade_type` at the database layer.

Alternative: CHECK constraint via raw migration:

```
ALTER TABLE fee_transfers
ADD CONSTRAINT chk_trade_type
CHECK (trade_type IN ('buy', 'sell', 'claim', 'auto_claim', 'copy_buy', 'copy_sell',
'limit_fill'));
```

Slightly less ergonomic but allows the column to remain `text` (compatible with existing rows that may already have invalid values, which would block the enum migration until backfilled).

Whichever path is chosen, audit existing data first:



```
SELECT DISTINCT trade_type FROM fee_transfers
WHERE trade_type NOT IN ('buy', 'sell', 'claim', 'auto_claim', 'copy_buy', 'copy_sell',
'limit_fill');
```

For each unexpected value, decide whether to backfill to `'limit_fill'` (known typo) or extend the union (new category). Migrate before adding the constraint.

[L-19] `isActive` not validated in `createUserPromo` or `claimPromo` after admin deactivation

Description

`findActivePromoByRefCode` correctly filters on `isActive = true` when resolving a KOL reference code to a promotion:

```
// packages/database/src/services/kol-promo.ts:59-75
async findActivePromoByRefCode(refCode: string): Promise<KolPromo | null> {
  try {
    const result = await this.db.query.kolPromos.findFirst({
      where: (kp, { eq, and, lt }) =>
        and(
          eq(kp.refCode, refCode),
          eq(kp.isActive, true),
          lt(kp.usedSlots, kp.availableSlots)
        ),
    });
    return result ? this.mapRowToKolPromo(result) : null;
  }
}
```

But the resolved `kolPromoId` is cached in session state (`ctx.session.pendingKolPromo.kolPromoId`) and consumed later during wallet creation. The two state-mutating steps that follow (slot burn and claim) do not re-check `isActive`. An admin who deactivates a promo between these steps cannot actually stop disbursements.

`createUserPromo` increments `usedSlots` with a WHERE clause that gates only on remaining slot capacity:

```
// kol-promo.ts:80-132 (createUserPromo)
return await this.db.transaction(async (tx) => {
  const updated = await tx.update(kolPromos)
    .set({
      usedSlots: sql`${kolPromos.usedSlots} + 1`,
      updatedAt: new Date(),
    })
    .where(and(
      eq(kolPromos.id, kolPromoId),
      sql`${kolPromos.usedSlots} < ${kolPromos.availableSlots}`
      // ← no eq(kolPromos.isActive, true)
    ))
    .returning({ id: kolPromos.id });

  if (updated.length === 0) {
    throw new DatabaseError('No available promo slots remaining');
  }
});
```



```
    }  
    ...  
    const [newUserPromo] = await tx.insert(userPromos).values({  
      userId,  
      kolPromoId,  
      status: 'pending',  
    }).onConflictDoNothing().returning();  
    ...  
  });
```

`claimPromo` validates the promo exists and the amount matches, but does not check `kolPromo.isActive` :

```
// kol-promo.ts:172-223 (claimPromo)  
const kolPromo = await this.db.query.kolPromos.findFirst({  
  where: (kp, { eq }) => eq(kp.id, userPromo.kolPromoId),  
});  
if (!kolPromo) {  
  throw new DatabaseError('Associated KOL promo not found');  
}  
if (amount !== kolPromo.claimableAmount) {  
  throw new DatabaseError(`Claim amount ${amount} does not match promo claimable amount $  
${kolPromo.claimableAmount}`);  
}  
// ← no "if (!kolPromo.isActive) throw ..."  
...  
const [updated] = await this.db.update(userPromos)  
  .set({  
    status: 'claimed',  
    claimedAt: now,  
    unlockAt,  
    claimedAmount: amount,  
    claimTxHash: txHash,  
    updatedAt: now,  
  })  
  .where(and(eq(userPromos.id, userPromoId), eq(userPromos.status, 'processing')))  
  .returning();
```

Exploit path:

1. User visits a KOL deep link. `findActivePromoByRefCode('KOL1')` returns the active promo. Session stores `kolPromoId` .
2. Admin deactivates: `UPDATE kol_promos SET is_active = false WHERE id = kolPromoId` . The intent is to stop disbursements on this campaign (budget exhausted, KOL terminated, fraud detected).
3. User completes wallet creation. `createUserPromo(userId, kolPromoId)` runs. The transaction's WHERE clause has no `isActive` check; the slot increment succeeds, `userPromos` row is inserted with `status='pending'` , and a slot of treasury budget is now earmarked for a deactivated campaign.
4. User clicks "Claim" on the promo. `claimPromo` validates existence and amount but does not check `isActive` . The row transitions to `claimed` , and the claim worker sends the user `claimableAmount` USDC from the master Safe.



This is two independent bugs with the same root cause, exploitable through the same entry point. A fix to only one of them still leaves the other as an unintended disbursement channel.

Affected files

- `packages/database/src/services/kol-promo.ts:80-132` : `createUserPromo` slot-burn transaction omits `isActive` from its WHERE clause.
- `packages/database/src/services/kol-promo.ts:172-223` : `claimPromo` omits `isActive` validation before the state transition.
- `packages/database/src/services/kol-promo.ts:59-75` : `findActivePromoByRefCode` filters on `isActive` correctly; the downstream mutators do not.

Impact: Medium

Admin deactivation of a KOL promo is silently ignored by both the slot-burn and the claim paths. Every user who resolved the promo ref code before the deactivation (session state persists for the length of the Telegram session) can proceed through wallet creation and claim `claimableAmount` USDC from the master Safe, draining the campaign's budget past the admin's intended kill moment.

The budget leak per user is bounded by `claimableAmount` (default \$20) times `availableSlots - usedSlots` at the moment of deactivation. In practice, the window of concurrent in-flight claims after a deactivation can be hundreds to thousands of users depending on traffic, and each successful claim is real treasury USDC.

Secondary: the slot burn goes through even when the campaign is deactivated, so `usedSlots` keeps climbing on a disabled row. Reporting that groups by `isActive` reports misleading totals.

Recommendations

Add `isActive` checks in both mutators.

In `createUserPromo`, add `eq(kolPromos.isActive, true)` to the WHERE clause of the slot-burn UPDATE:

```
const updated = await tx.update(kolPromos)
  .set({
    usedSlots: sql`${kolPromos.usedSlots} + 1`,
    updatedAt: new Date(),
  })
  .where(and(
    eq(kolPromos.id, kolPromoId),
    eq(kolPromos.isActive, true),
    sql`${kolPromos.usedSlots} < ${kolPromos.availableSlots}`
  ))
  .returning({ id: kolPromos.id });
```



```
if (updated.length === 0) {
  throw new DatabaseError('No available promo slots or promo is no longer active');
}
```

In `claimPromo`, reject deactivated promotions before the status transition:

```
const kolPromo = await this.db.query.kolPromos.findFirst({
  where: (kp, { eq }) => eq(kp.id, userPromo.kolPromoId),
});
if (!kolPromo) throw new DatabaseError('Associated KOL promo not found');
if (!kolPromo.isActive) throw new DatabaseError('KOL promo is no longer active');
if (amount !== kolPromo.claimableAmount) throw new DatabaseError(...);
```

Both changes are necessary: fixing only `createUserPromo` still allows users whose row was created before deactivation to claim. Fixing only `claimPromo` still lets a slot burn on a deactivated promo, leaving the ledger in an inconsistent state.

Belt-and-braces: when an admin flips `isActive` to false, run a one-shot `UPDATE user_promos SET status='revoked' WHERE kolPromoId = ? AND status='pending'` so pending rows cannot later transition to `processing` or `claimed`.

[L-20] Unbounded `markWithdrawn` function marks all credited commissions as withdrawn

Description

`CommissionService.markWithdrawn` (`commissions.ts`) sweeps all `credited` commission rows for a beneficiary into `withdrawn` status, with no bounds on which rows are included:

```
await this.db.update(referralCommissions)
  .set({ status: 'withdrawn', txHash, transferredAt: new Date() })
  .where(and(
    eq(referralCommissions.beneficiaryId, beneficiaryId),
    eq(referralCommissions.status, 'credited')
  ));
```

The withdrawal flow has two steps separated by an on-chain transaction:

1. `resetPendingCommission` (`bot-v2/index.ts`) zeroes `pending_commission` and captures the previous amount.
2. `markWithdrawn` (in `referral-withdraw.worker.ts`) is called after the on-chain USDC transfer completes.

Between these two steps, new trades can complete and insert fresh `credited` commission rows via the fee-transfer worker (`trading-utils.ts`). These new rows were not included in the captured amount sent on-chain, but `markWithdrawn` sweeps them into `withdrawn` status along with the legitimate ones.



As a result, the commissions which arrive mid-window are marked as paid by `txHash` but are never included in the on-chain payout amount. The commission ledger shows them as withdrawn while the user was never paid for them, leading to fund loss for the user.

Recommendations

Scope `markWithdrawn` to a specific set of commission IDs captured atomically with the balance snapshot, rather than sweeping all `credited` rows by `beneficiary Id`.

[L-21] Case-sensitive `findBySafeAddress` fails on checksummed addresses in database

Description

`UserService.findBySafeAddress` in `packages/database/src/services/users.ts` performs an exact-match lookup with no case normalization:

```
async findBySafeAddress(safeAddress: string): Promise<User | null> {
  return withRetry(async () => {
    const result = await this.db
      .select()
      .from(users)
      .where(and(
        eq(users.gnosisSafeAddress, safeAddress), // case-sensitive text =
        isNull(users.deletedAt)
      ))
      .limit(1);
    return result[0] ? this.mapRowToUser(result[0]) : null;
  }, 'Failed to find user by wallet');
}
```

PostgreSQL's `eq` on `text` columns is case-sensitive. Two different representations of the same address `0xAbCdEfxYZ` (EIP-55 checksummed) vs `0xabcdefxyz` (lowercased) do not match and make the matching inconsistent with the intended use case.

The address is stored in the DB via `updateWalletInfo` (`packages/database/src/services/users.ts`), which writes `data.gnosisSafeAddress` as is. The upstream `deploySafeForUser` returns a checksummed address from Solidity's `computeProxyAddress()` (ethers.js formats addresses with EIP-55 mixed-case by default). Therefore, the DB stores **checksummed** addresses like `0xAbCdEfxYZ`.

However, callers of `findBySafeAddress` pass **lowercased** addresses. The copy-trade preprocessor at `packages/queue/src/nats-consumers/copy-trade-preprocessor.consumer.ts` lowercases the on-chain event's maker address (`maker.toLowerCase()`) before calling `findBySafeAddress`, resulting in silent failure as the result is always `null`.



This silent failure has a concrete impact on the `copy-trade self-follow` guard. Both preprocessors use `findBySafeAddress` to resolve the leader's on-chain address back to an internal user, then filter out self-follows:

```
// copy-trade-preprocessor.consumer.ts:498-507
const leaderUser = await userService.findBySafeAddress(leaderAddress); // always null (case mismatch)

const eligibleFollowers = leaderUser?.telegramId // undefined
(null?.telegramId)
  ? followers.filter((f) => f.followerTelegramId !== leaderUser.telegramId)
  : followers; // unfiltered – self-follows pass through
```

Because `findBySafeAddress` always returns `null` for internal users (checksummed DB vs lowercased query), `leaderUser` is always `null`, `leaderUser?.telegramId` is always `undefined`, and the ternary falls through to `followers` and shows the **unfiltered** list. The self-follow filter is effectively dead code. This means a user who self-follows is never filtered out at the preprocessor level.

Recommendations

Normalize to lowercase in the query:

```
// copy-trade-preprocessor.consumer.ts
const leaderUser = await userService.findBySafeAddress(leaderAddress); // always null (case mismatch)

const eligibleFollowers = leaderUser?.telegramId // undefined
(null?.telegramId)
  ? followers.filter((f) => f.followerTelegramId !== leaderUser.telegramId)
  : followers; // unfiltered – self-follows pass through
```

[L-22] Relay execution accepts unverified safe address

Description

The relay helper and withdrawal worker authenticate a user and decrypt the user wallet, but the Safe address used for execution is supplied by the caller or queue payload and is never proven to match the owner-derived Safe for that private key.

Location:

- `packages/web3/src/relay-polymarket/helpers.ts`
- `packages/web3/src/relay-polymarket/RelayPolymarketSDK.ts`
- `packages/queue/src/workers/withdraw.worker.ts`
- `packages/queue/src/queues/trading.queue.ts`



Reachability / endpoint: Telegram withdrawal flow and any worker path that calls

`createRelaySDK()` or `setSafeAddress()`

Vulnerable flow:

The worker loads the user and decrypts the wallet from the database, then passes

`job.data.safeAddress` into `createRelaySDK()`. `createRelaySDK()` simply calls `sdk.setSafeAddress(options.safeAddress)`. The SDK never verifies that this Safe belongs to the private key owner via `computeProxyAddress()` or deployed Safe discovery before transfer execution.

Affected code:

```
export function createRelaySDK(options: CreateRelaySDKOptions): RelayPolymarketSDK {
  const sdk = new RelayPolymarketSDK({ wallet: options.privateKey, credentials:
options.credentials });
  sdk.setSafeAddress(options.safeAddress);
  return sdk;
}
```

```
const { telegramId, destinationAddress, amount, safeAddress } = job.data;
const privateKey = services.walletManager.getDecryptedPrivateKey(wallet);
const sdk = createRelaySDK({ privateKey, safeAddress, credentials: ... });
await sdk.transferUsdc({ to: destinationAddress, amount: amountInUnits });
```

Authentication and authorization are split across different trust sources. The private key is authoritative. The queue payload is not. Under normal operation, the `safeAddress` originates from `user.gnosisSafeAddress` in the database, flows through the Telegram session into the queue job, and will be correct. However, the worker never re-derives or re-validates the Safe against the private key owner. Exploitation requires the ability to mutate a BullMQ job payload — achievable through direct Redis access, the unauthenticated NATS/ admin control planes, or any future queue-injection vector. Once a payload is mutated, the worker will execute a USDC transfer from an arbitrary Safe using the victim's decrypted key.

Impact:

- Unauthorized fund movement (requires queue-payload manipulation, enabled by unauthenticated control planes)
- Wrong-wallet execution if session or database state becomes stale between enqueue and processing
- State corruption between user and wallet identity

Recommendations

Derive the expected Safe from the private key owner every time privileged relay execution is prepared. Reject any caller-supplied Safe that does not exactly match the derived and deployed Safe.



```
const sdk = new RelayPolymarketSDK({ wallet: privateKey, credentials });
const expectedSafe = await sdk.getDeployedSafeAddress() ?? await sdk.computeProxyAddress();
if (expectedSafe.toLowerCase() !== user.gnosisSafeAddress!.toLowerCase()) {
  throw new Error('Authoritative Safe mismatch');
}
sdk.setSafeAddress(expectedSafe);
```